

Problem Statement

The objective is to analyze and identify the key factors that influence graduate admissions.
By leveraging machine learning models, we aim to:

- Determine which factors contribute most significantly to the admission decision.
- Understand how these factors are interrelated.
- Develop a predictive model that estimates an applicant's probability of admission based on their profile.

This analysis will help *Jamboree* provide data-driven guidance to students on improving their chances of securing admission into graduate programs.

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.metrics import r2_score, mean_squared_error
```

```
In [2]: df = pd.read_csv("https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/001/839/original/Jamboree_Admission.csv")
df
```

Out[2]:

	Serial No.	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	1	337	118	4	4.5	4.5	9.65	1	0.92
1	2	324	107	4	4.0	4.5	8.87	1	0.76
2	3	316	104	3	3.0	3.5	8.00	1	0.72
3	4	322	110	3	3.5	2.5	8.67	1	0.80
4	5	314	103	2	2.0	3.0	8.21	0	0.65
...	...	...	...	...	...	...	...	...	...
495	496	332	108	5	4.5	4.0	9.02	1	0.87
496	497	337	117	5	5.0	5.0	9.87	1	0.96
497	498	330	120	5	4.5	5.0	9.56	1	0.93
498	499	312	103	4	4.0	5.0	8.43	0	0.73
499	500	327	113	4	4.5	4.5	9.04	0	0.84

500 rows × 9 columns

Basic Exploration

```
In [3]: # size of the dataframe

df.shape
```

Out[3]: (500, 9)

```
In [4]: # dropping the Serial No. column, which might create a problem in future

df = df.drop('Serial No.', axis=1)
df.head()
```

Out[4]:

	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
0	337	118	4	4.5	4.5	9.65	1	0.92
1	324	107	4	4.0	4.5	8.87	1	0.76
2	316	104	3	3.0	3.5	8.00	1	0.72
3	322	110	3	3.5	2.5	8.67	1	0.80
4	314	103	2	2.0	3.0	8.21	0	0.65

```
In [5]: # basic information of the dataframe

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   GRE Score       500 non-null   int64
1   TOEFL Score     500 non-null   int64
2   University Rating 500 non-null   int64
3   SOP             500 non-null   float64
4   LOR             500 non-null   float64
5   CGPA            500 non-null   float64
6   Research        500 non-null   int64
7   Chance of Admit  500 non-null   float64
dtypes: float64(4), int64(4)
memory usage: 31.4 KB
```

```
In [6]: # List of columns of the dataframe after dropping serial no.

df.columns
```

Out[6]: Index(['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR ', 'CGPA', 'Research', 'Chance of Admit '], dtype='object')

In [7]: *# checking missing values in the dataframe*

```
df.isna().sum()
```

Out[7]:

GRE Score	0
TOEFL Score	0
University Rating	0
SOP	0
LOR	0
CGPA	0
Research	0
Chance of Admit	0

dtype: int64

In [8]: *# checking the unique values*

```
df.nunique()
```

Out[8]:

GRE Score	49
TOEFL Score	29
University Rating	5
SOP	9
LOR	9
CGPA	184
Research	2
Chance of Admit	61

dtype: int64

Summary:

- Continuous → GRE, TOEFL, SOP, LOR, CGPA, Chance of Admit
- Categorical → University Rating, Research

In [9]: *# removing leading and trailing spaces from the column names. LOR had the leading space*

```
df.columns = df.columns.str.strip()
```

Exploratory Data Analysis

Univariate analysis of all variables to check the distribution

In [10]: *# checking the distribution of the categorical columns*

```
categorical_cols = ['University Rating', 'Research']

for column in categorical_cols:
    print(f"Value counts for column: {column}")
    print(df[column].value_counts())
    print("\n" + "-"*50 + "\n")
```

Value counts for column: University Rating

3	162
2	126
4	105
5	73
1	34

Name: University Rating, dtype: int64

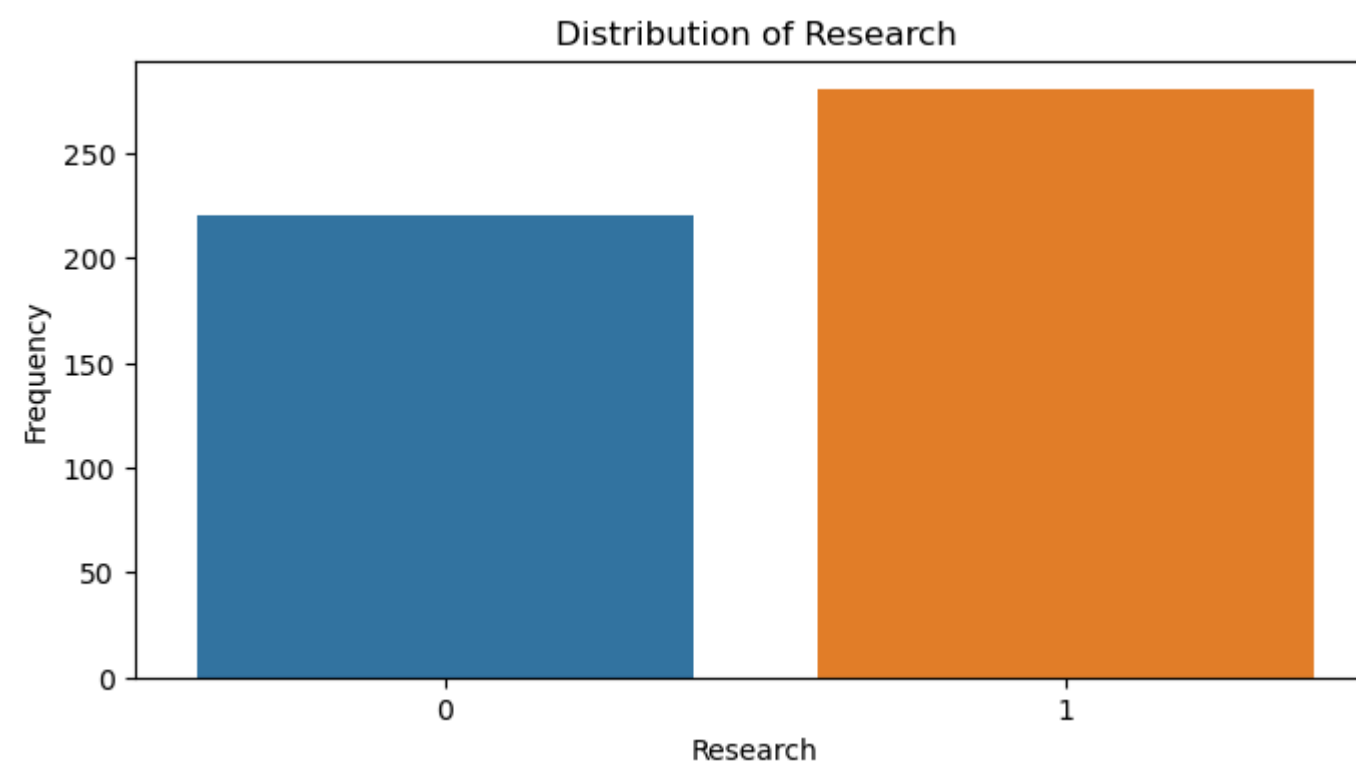
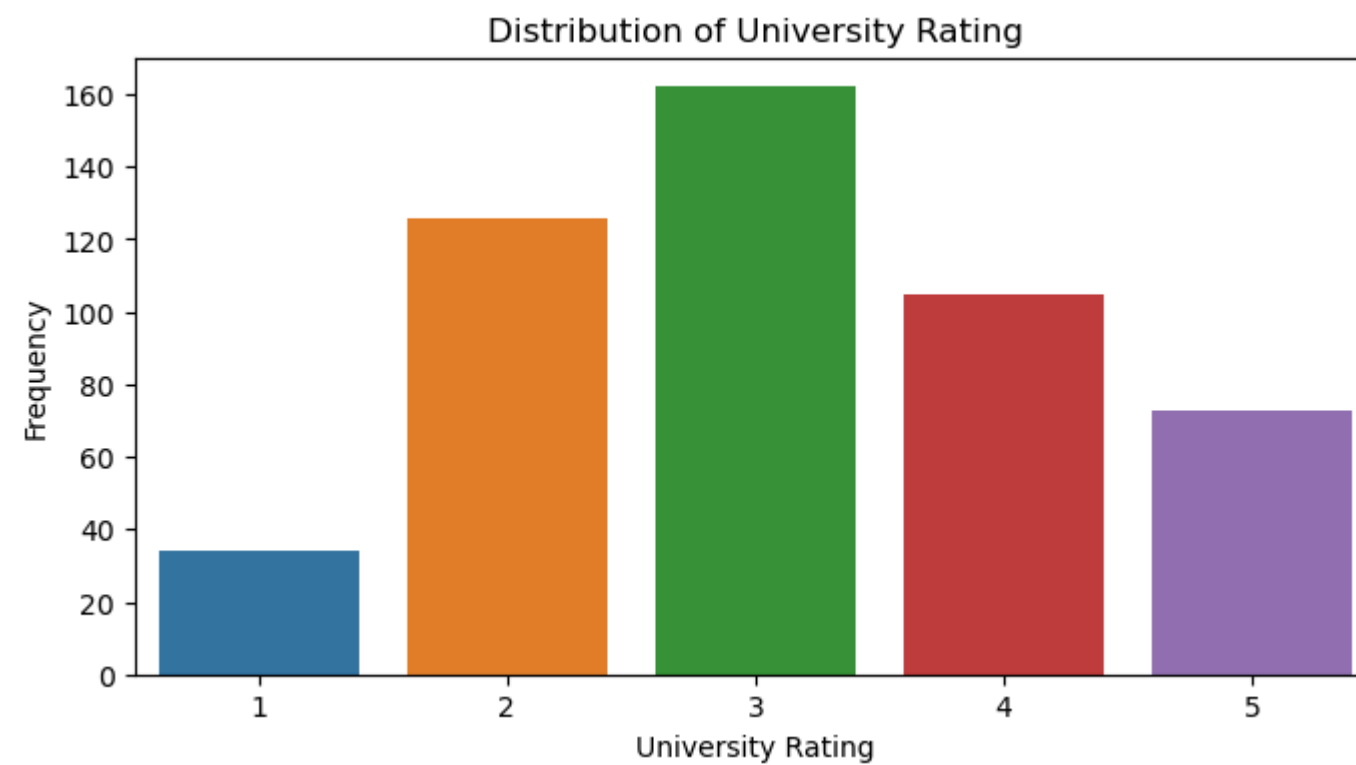
Value counts for column: Research

1	280
0	220

Name: Research, dtype: int64

In [11]: *# using countplot to visualise the distribution of categorical variables*

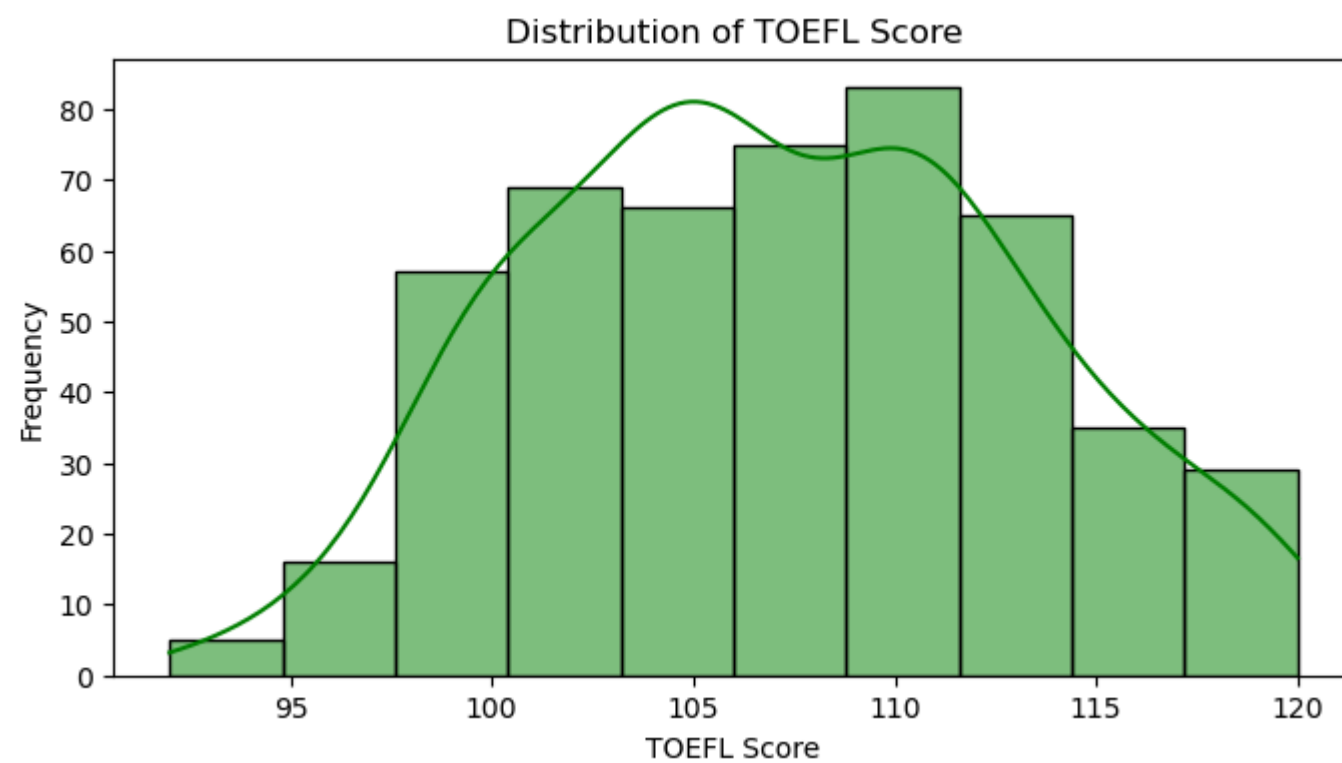
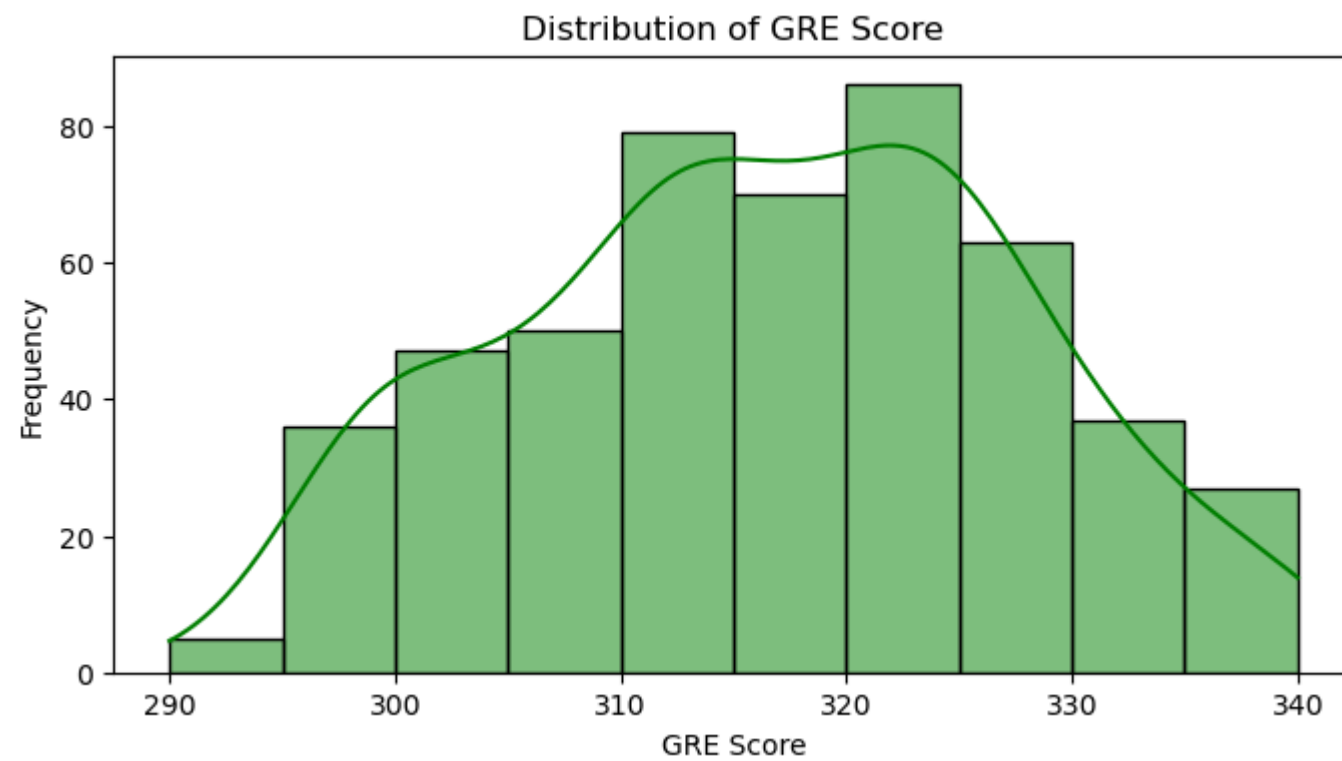
```
for column in categorical_cols:
    plt.figure(figsize = (8,4))
    sns.countplot(x = df[column])
    plt.title(f"Distribution of {column}")
    plt.ylabel("Frequency")
    plt.xlabel(column)
    #plt.grid(axis='y', line='--', alpha=0.5)
    plt.show()
```

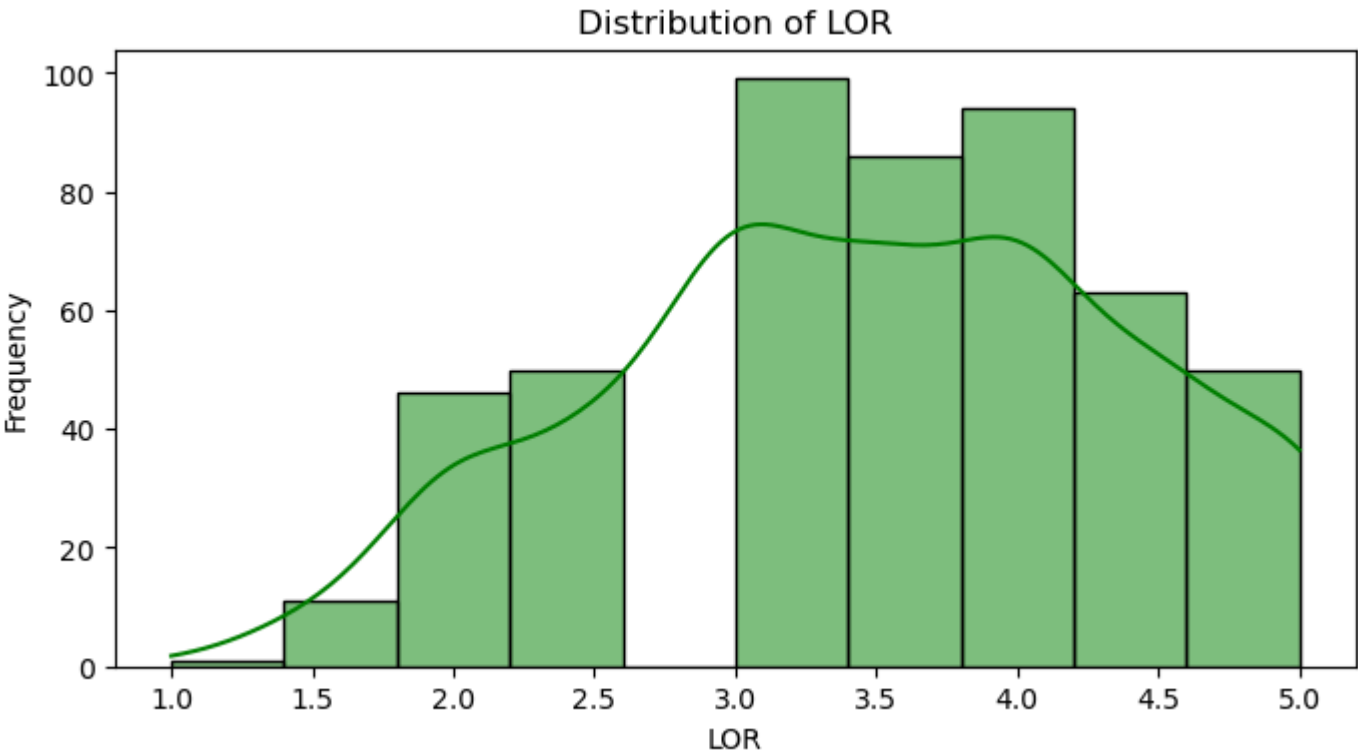
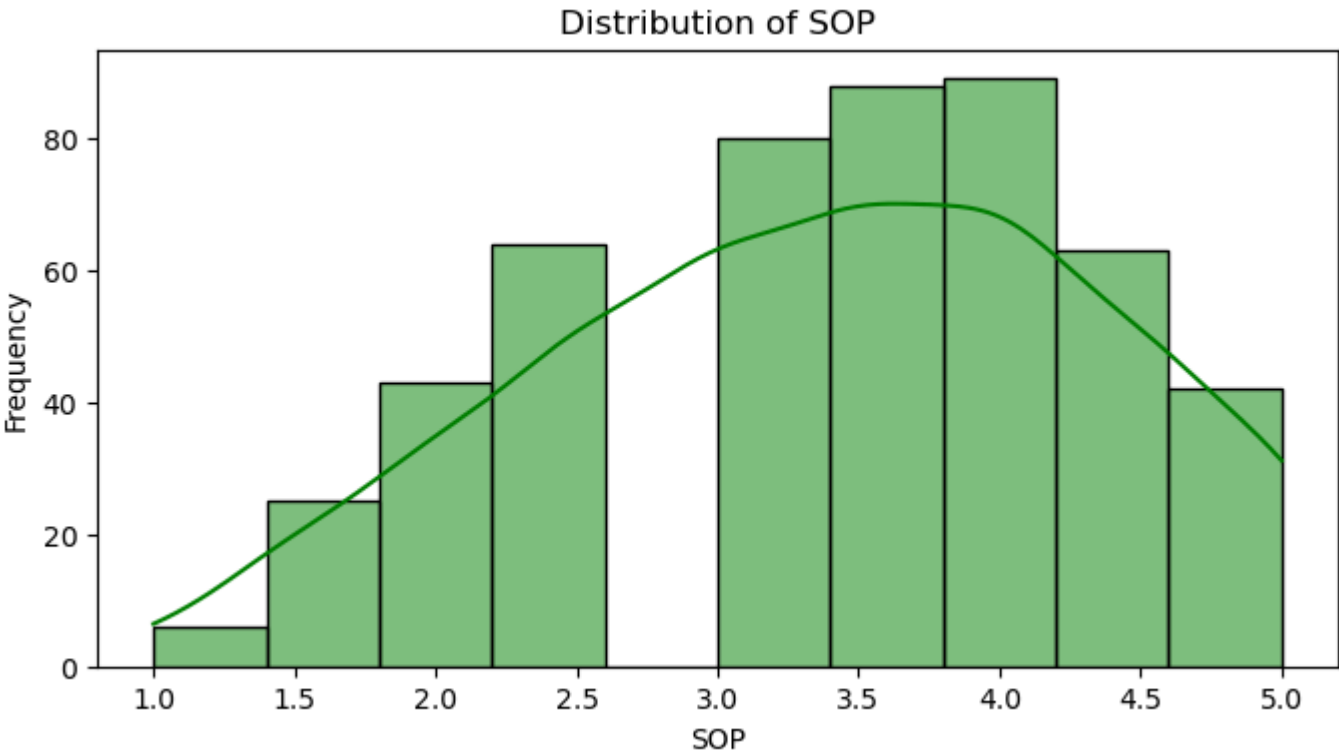


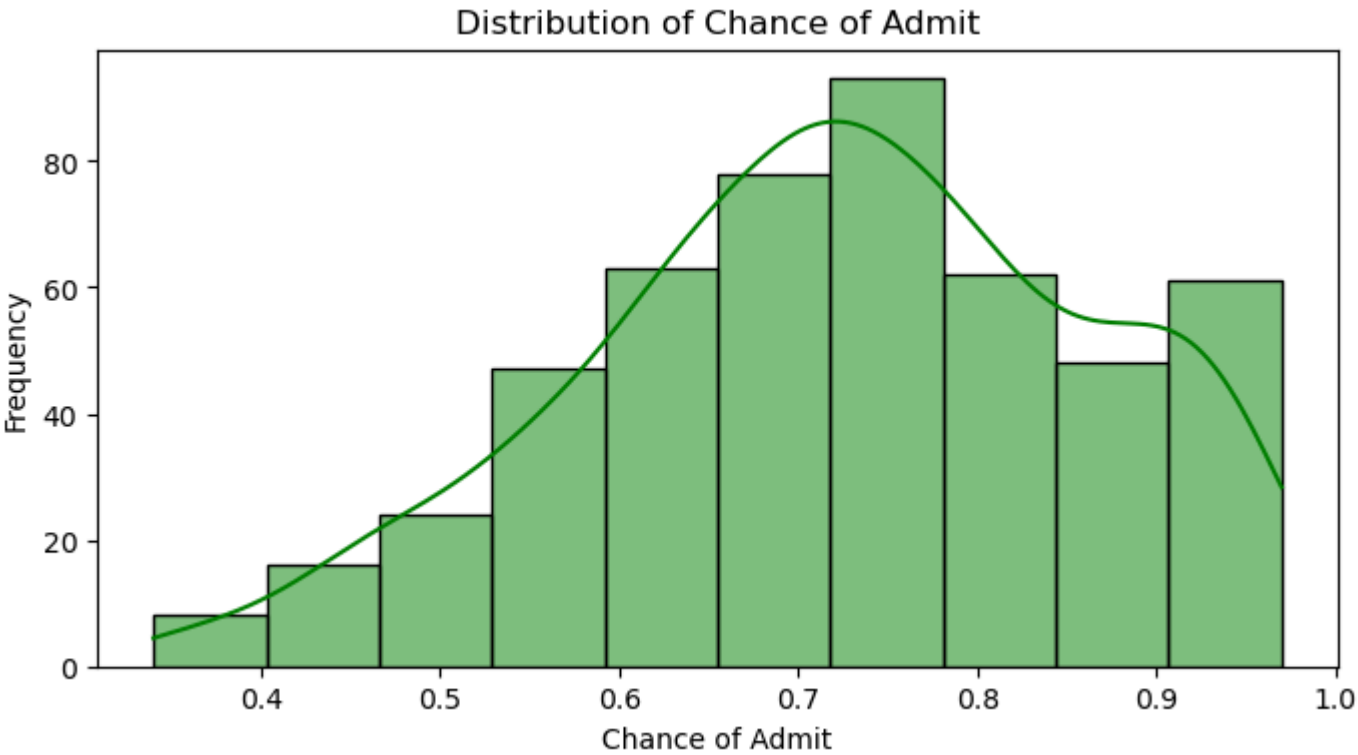
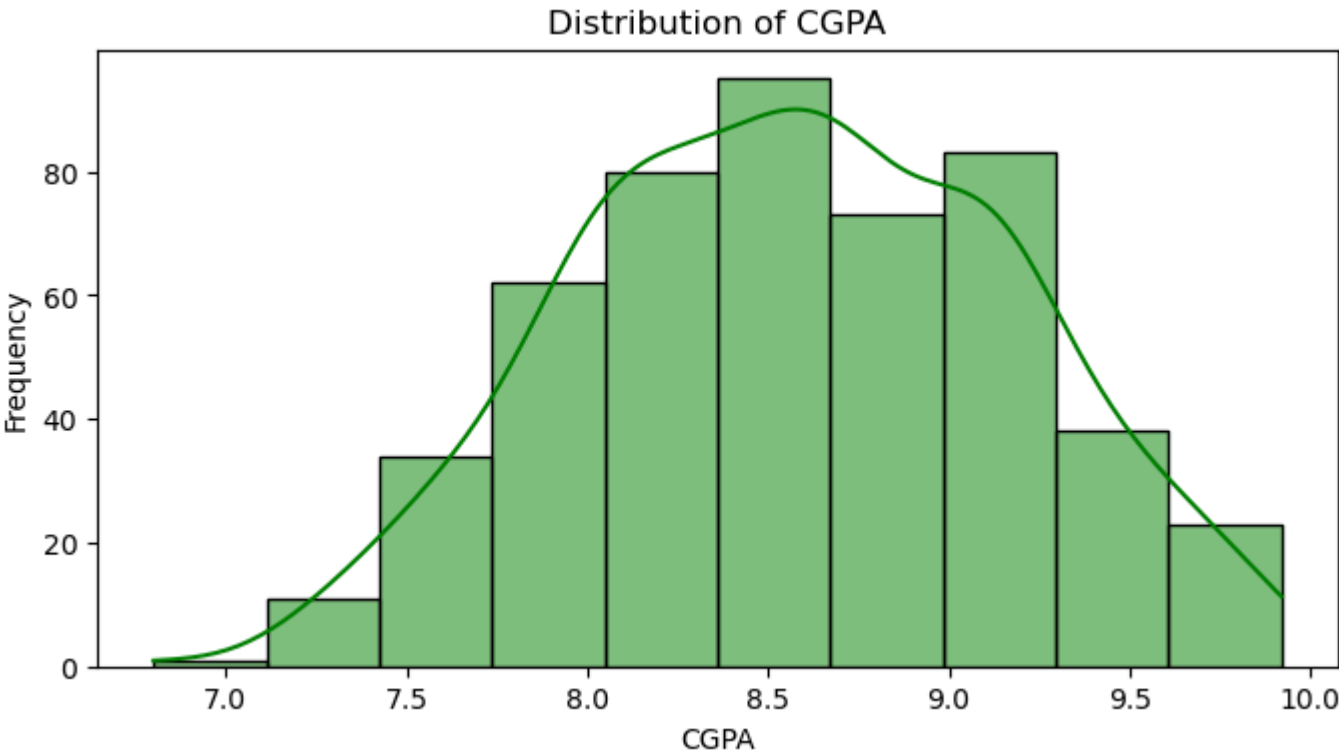
In [12]: *# checking the distribution of the numerical columns*

```
numerical_cols = ['GRE Score', 'TOEFL Score', 'SOP', 'LOR', 'CGPA', 'Chance of Admit']

for column in numerical_cols:
    plt.figure(figsize=(8,4))
    sns.histplot(df[column], kde=True, bins=10, color='green')
    plt.title(f"Distribution of {column}")
    plt.ylabel("Frequency")
    plt.xlabel(column)
    plt.show()
```

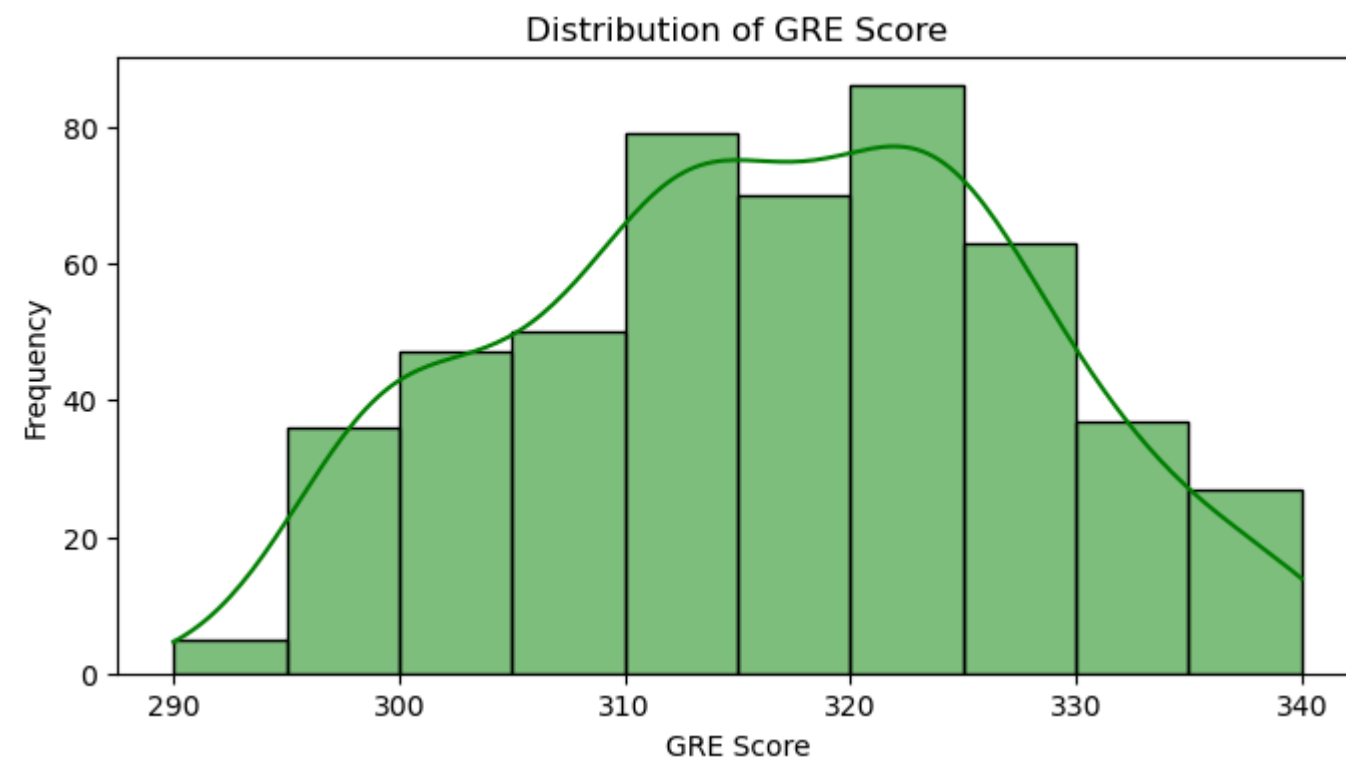


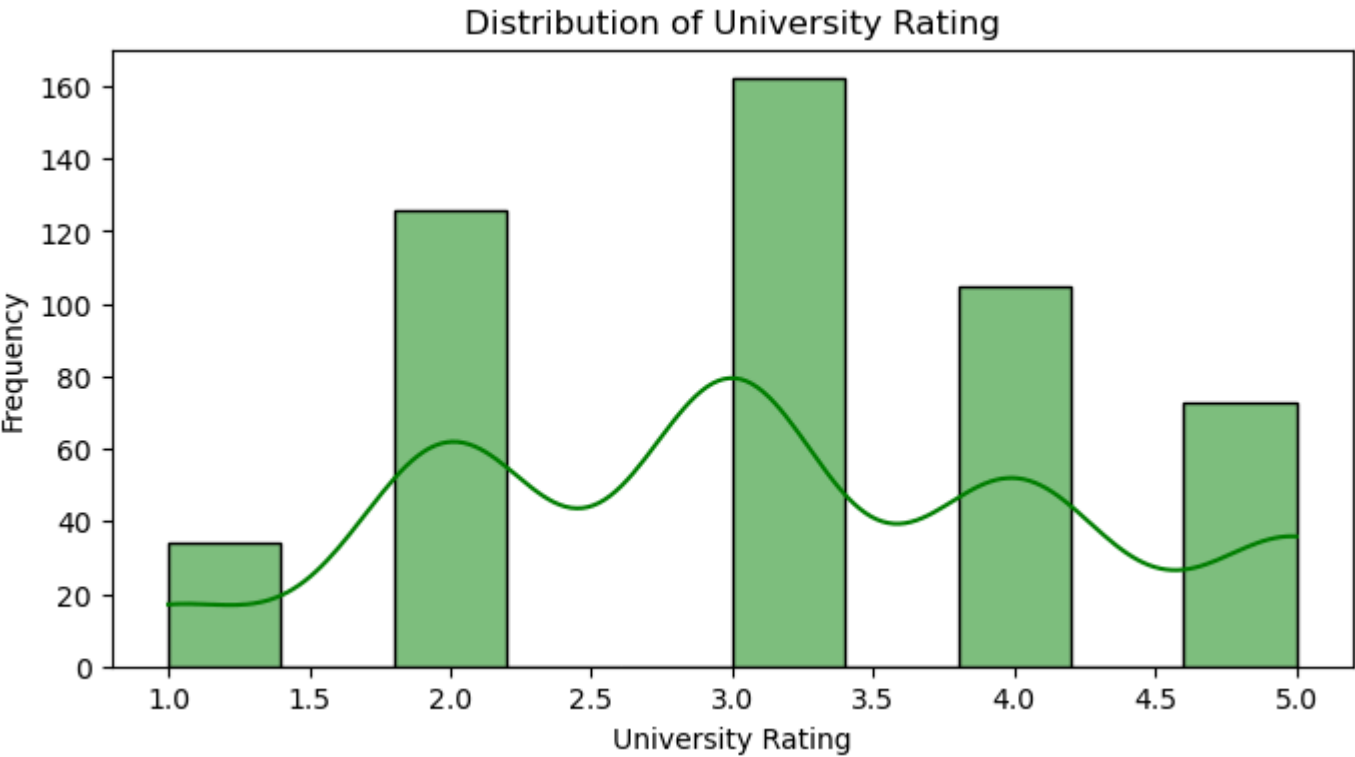
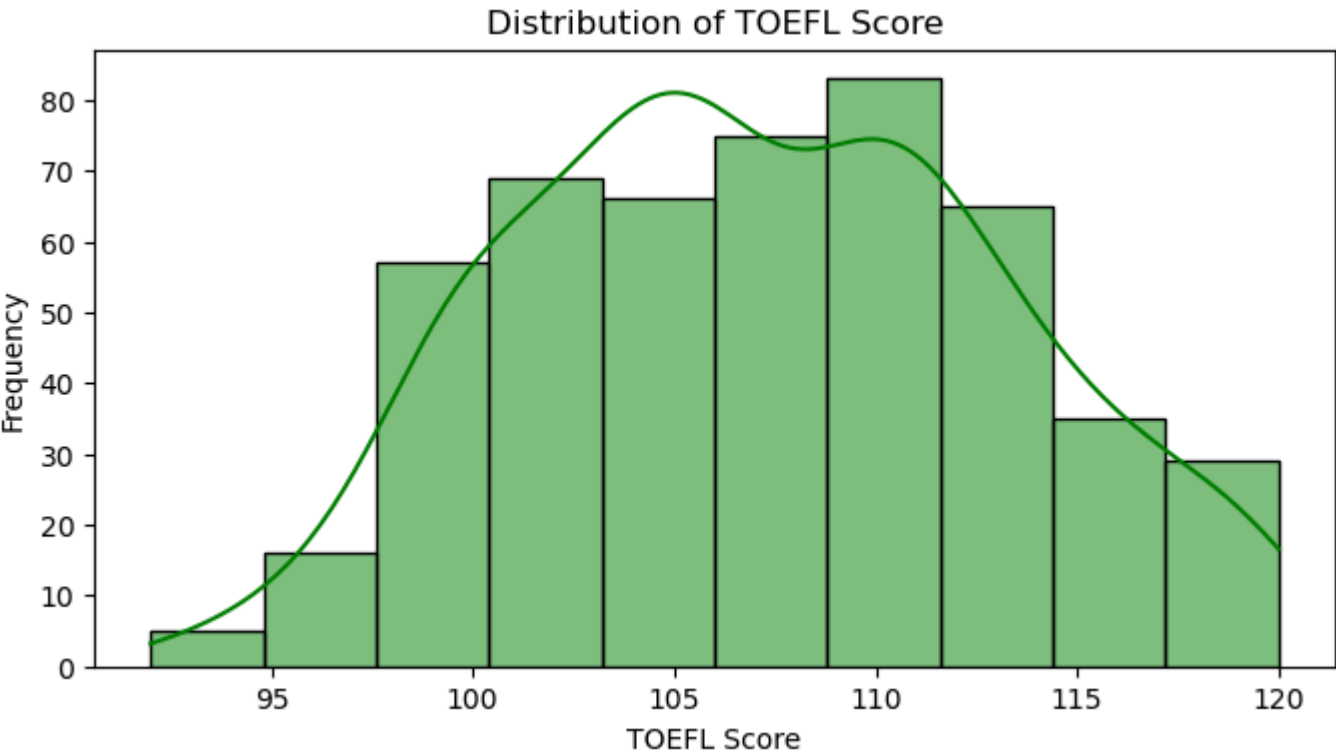


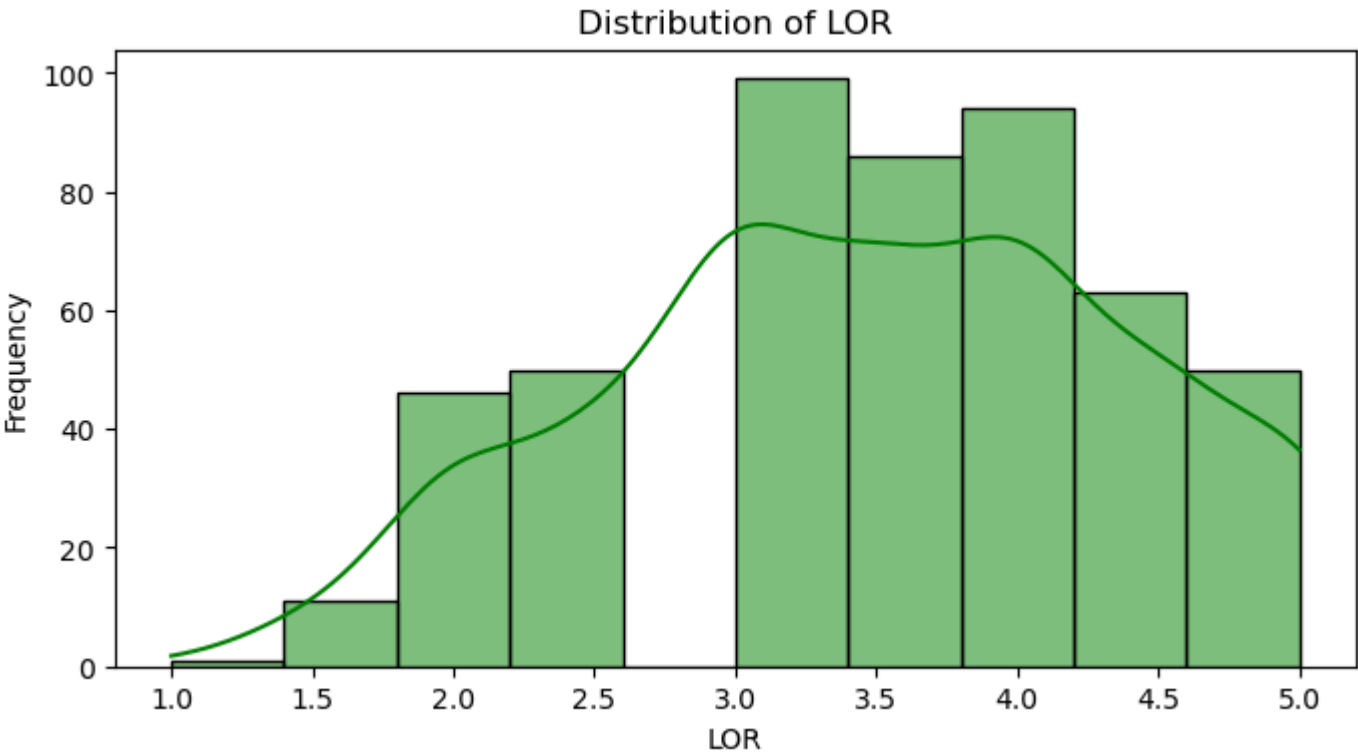
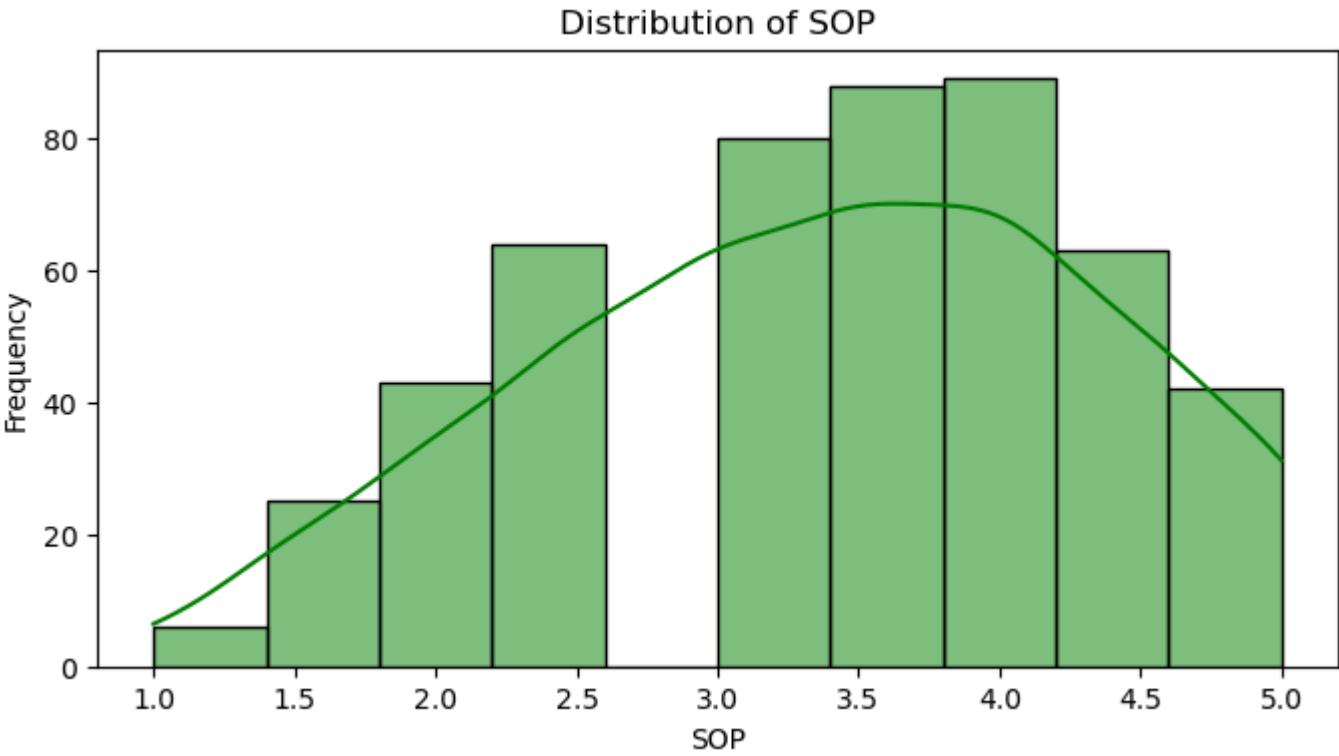


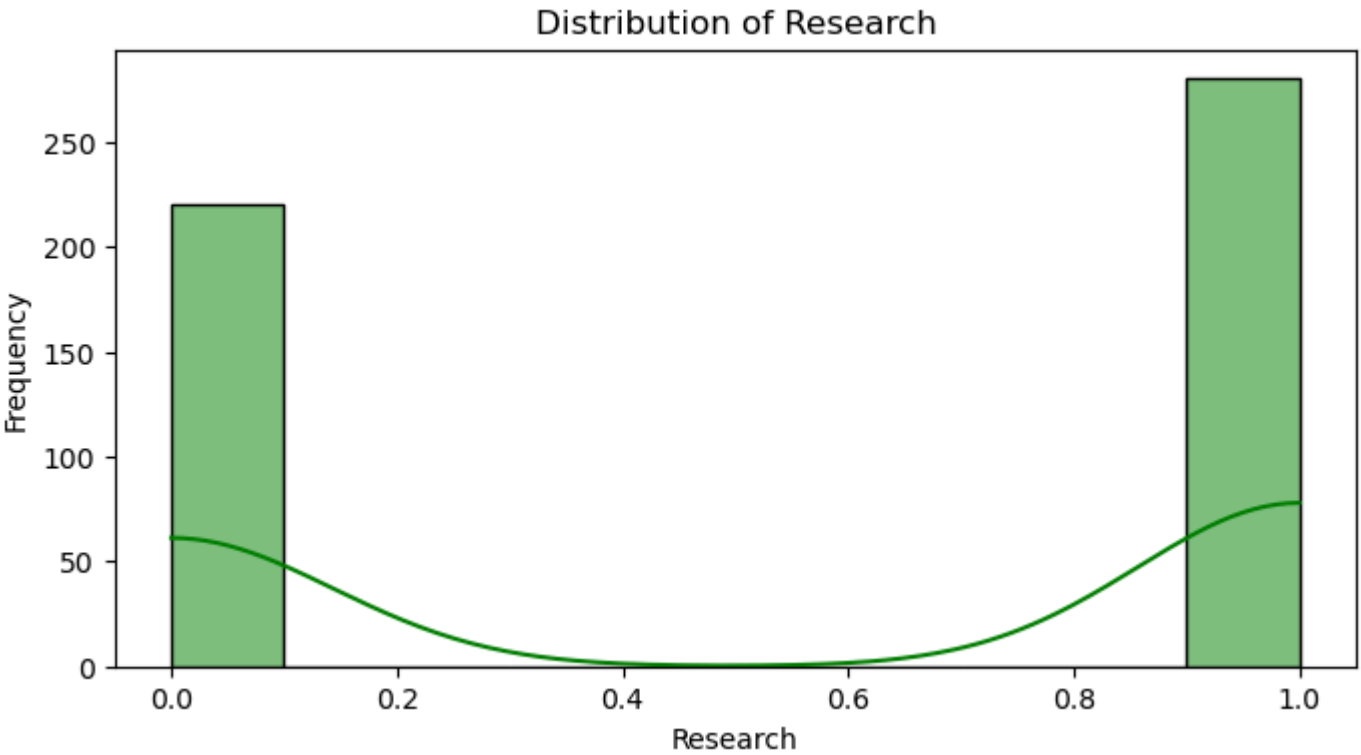
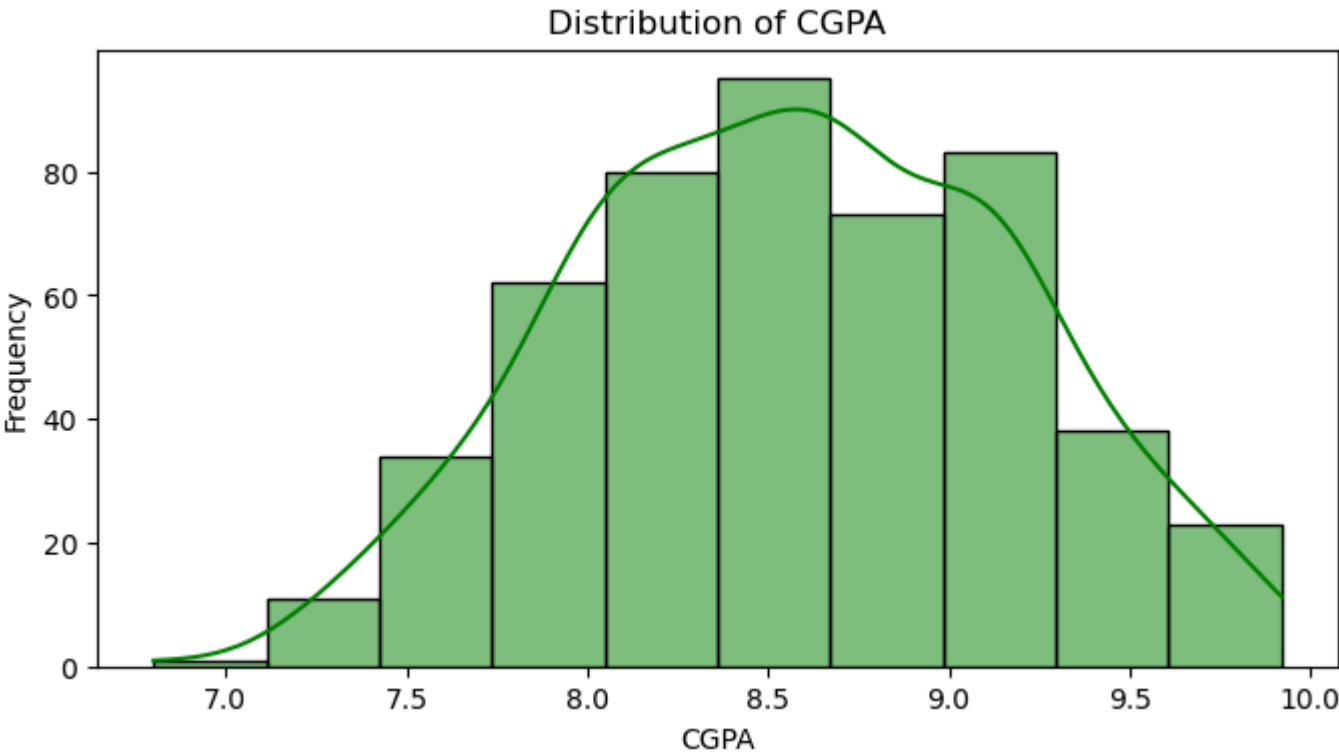
Checking diversity in Applicant profiles

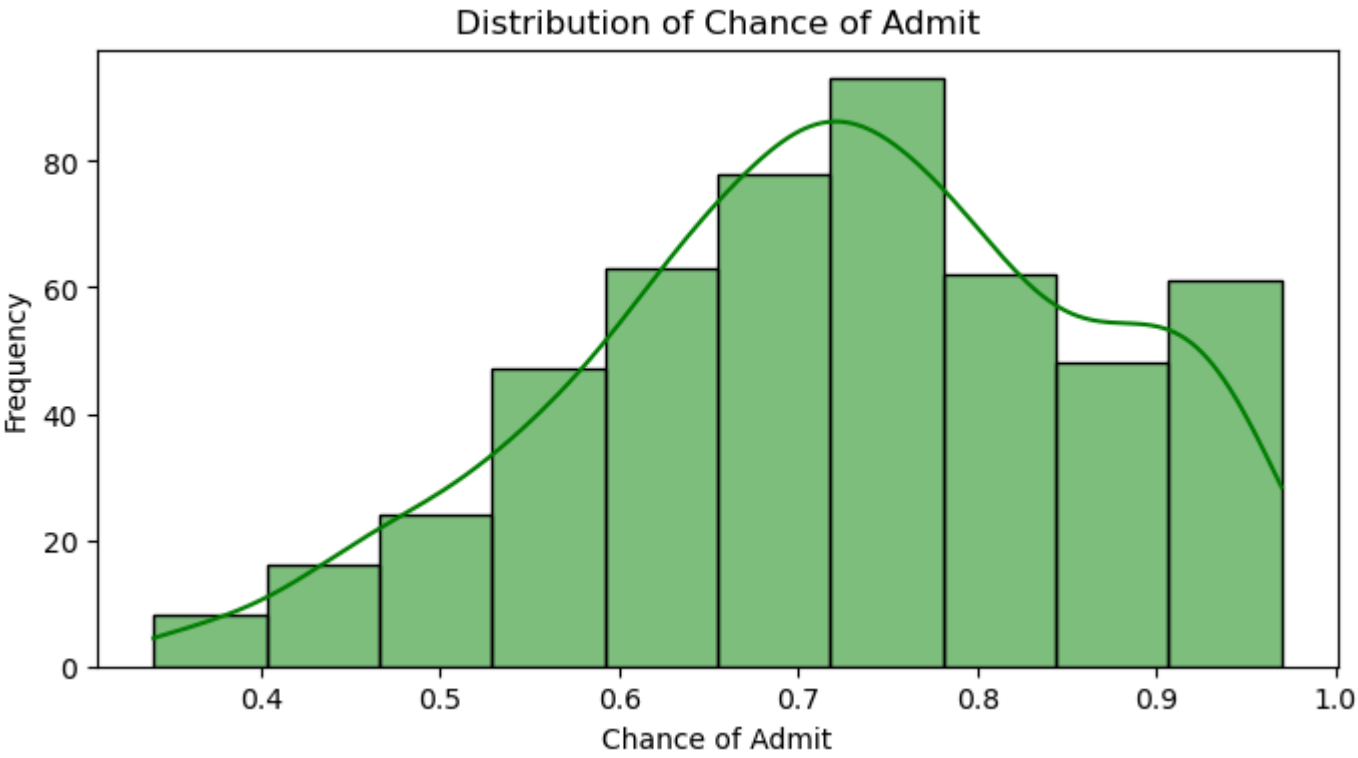
```
In [13]: features = ['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR', 'CGPA',  
                    'Research', 'Chance of Admit']  
  
for column in features:  
    plt.figure(figsize=(8,4))  
    sns.histplot(df[column], kde=True, bins=10, color='green')  
    plt.title(f"Distribution of {column}")  
    plt.ylabel("Frequency")  
    plt.xlabel(column)  
    plt.show()
```











In [14]: *# Summary statistics of the dataframe*

```
df.describe().round(2)
```

Out[14]:

	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research	Chance of Admit
count	500.00	500.00	500.00	500.00	500.00	500.00	500.00	500.00
mean	316.47	107.19	3.11	3.37	3.48	8.58	0.56	0.72
std	11.30	6.08	1.14	0.99	0.93	0.60	0.50	0.14
min	290.00	92.00	1.00	1.00	1.00	6.80	0.00	0.34
25%	308.00	103.00	2.00	2.50	3.00	8.13	0.00	0.63
50%	317.00	107.00	3.00	3.50	3.50	8.56	1.00	0.72
75%	325.00	112.00	4.00	4.00	4.00	9.04	1.00	0.82
max	340.00	120.00	5.00	5.00	5.00	9.92	1.00	0.97

Summarized Insights

- **GRE & TOEFL:** Scores are tightly clustered around the mean (GRE $\approx$ 316, TOEFL $\approx$ 107) with low variability, indicating most applicants perform well academically.
- **University Rating, SOP & LOR:** Majority fall in the mid-range (2–4), suggesting average-to-good ratings are common.
- **CGPA:** High overall (mean $\approx$ 8.6), with most students having strong academic records.
- **Research:** ~56% of applicants have research experience, making it an important differentiator.
- **Chance of Admit:** Centered around 0.72, with most applicants having a moderate-to-high probability of admission.

Key Takeaways

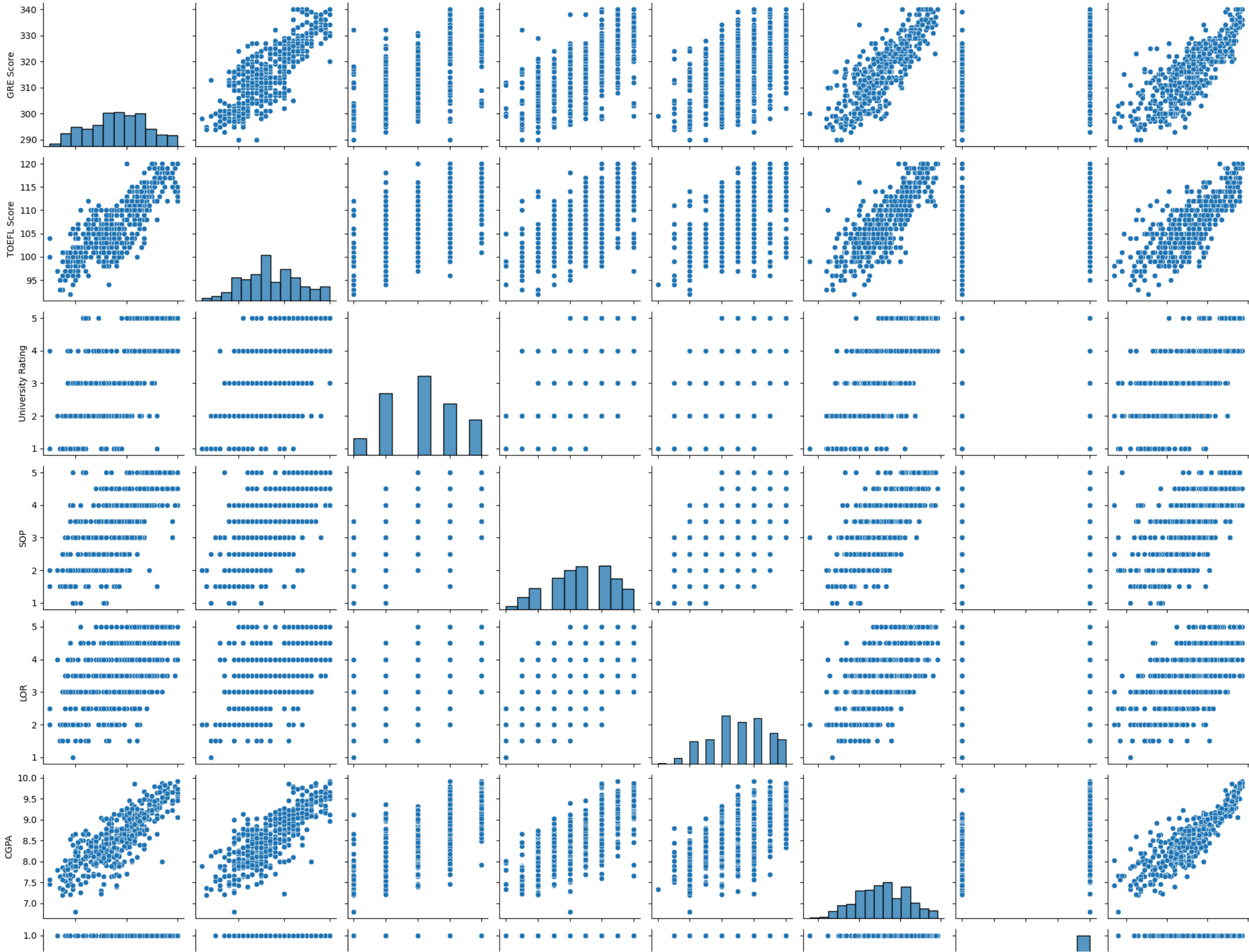
- Academic scores are strong for most applicants → less variance in GRE/TOEFL/CGPA.
- Qualitative factors (SOP, LOR, Research, University Rating) likely play a bigger role in distinguishing candidates.
- Dataset leans toward well-qualified applicants, so predictions will be more relevant for competitive profiles.

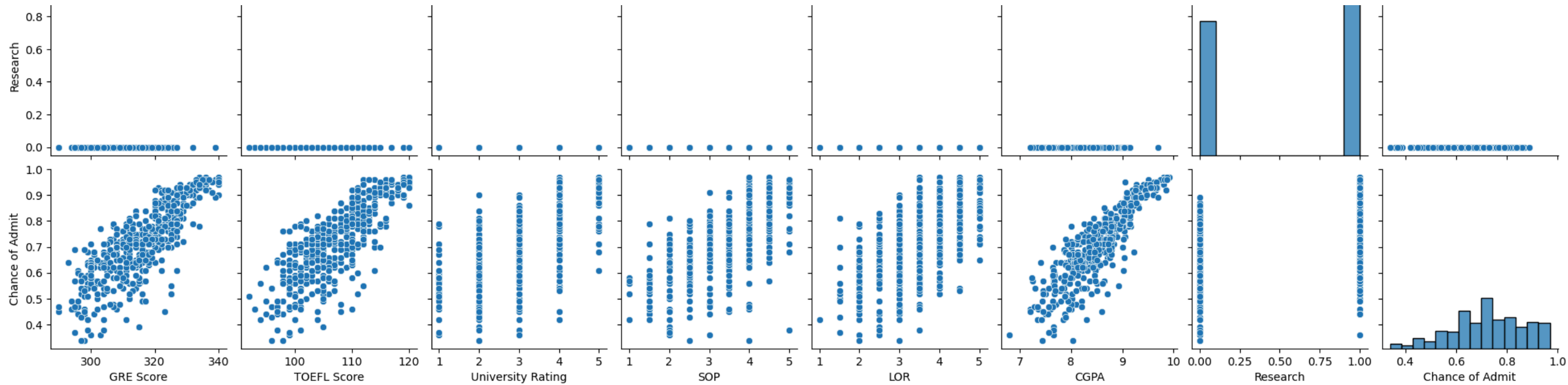
Bivariate Analysis between features

In [15]: *# checking the distribution between the features using pairplot*

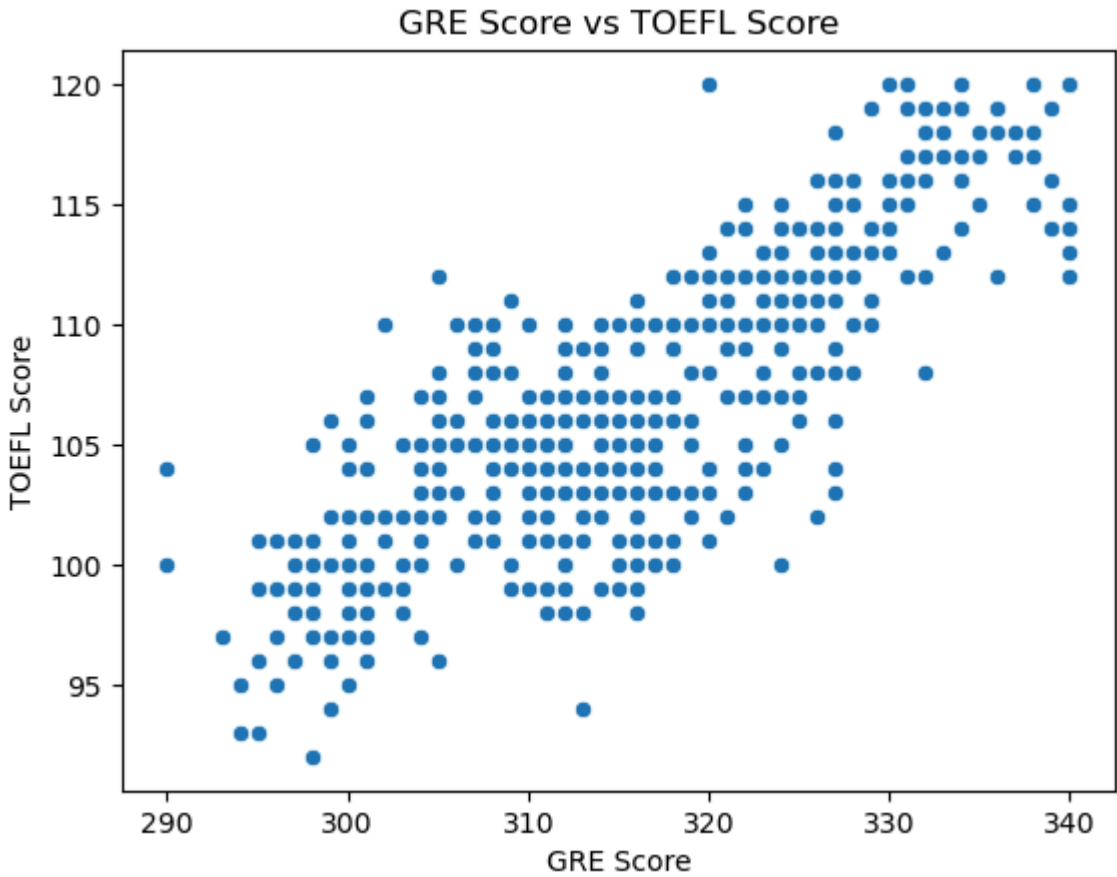
```
sns.pairplot(df)
```

Out[15]: <seaborn.axisgrid.PairGrid at 0x1d4a523be50>



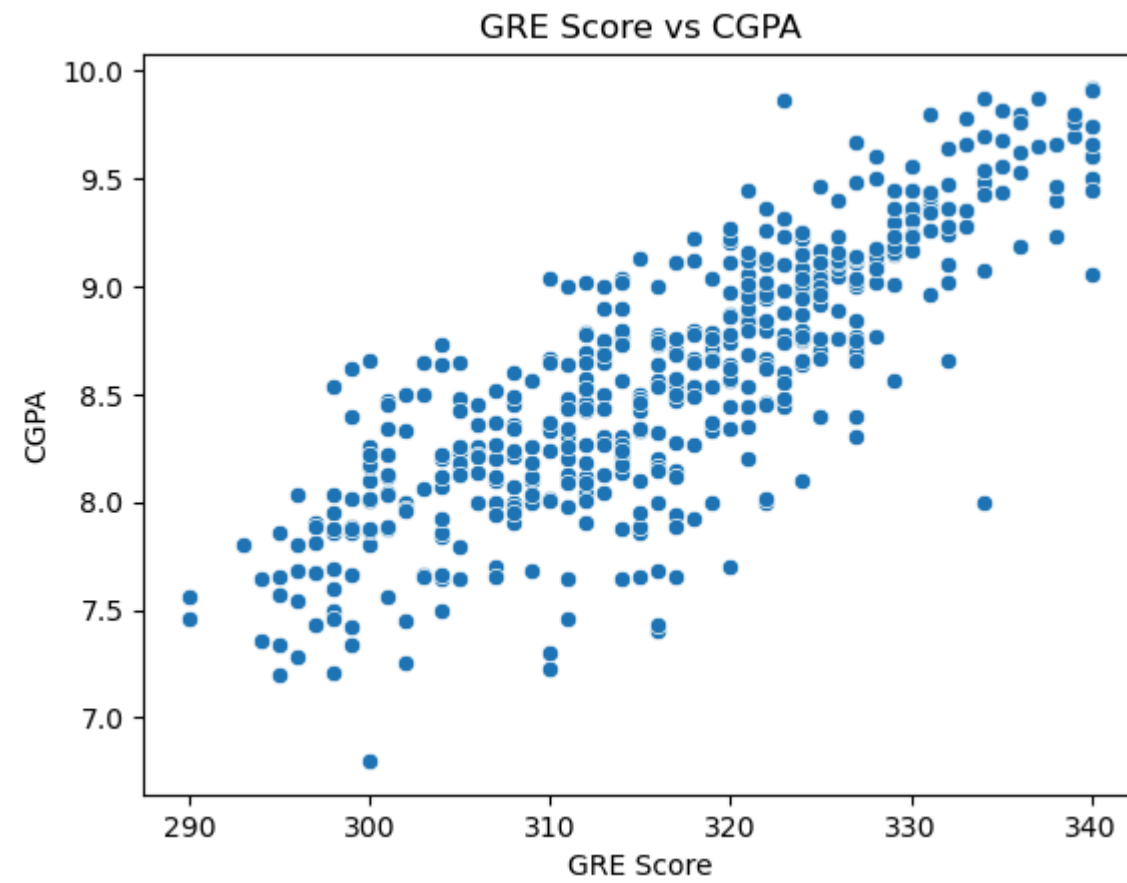


```
In [16]: # GRE Score Vs TOEFL
sns.scatterplot(data=df, x='GRE Score', y='TOEFL Score')
plt.title("GRE Score vs TOEFL Score")
plt.xlabel("GRE Score")
plt.ylabel("TOEFL Score")
plt.show()
```

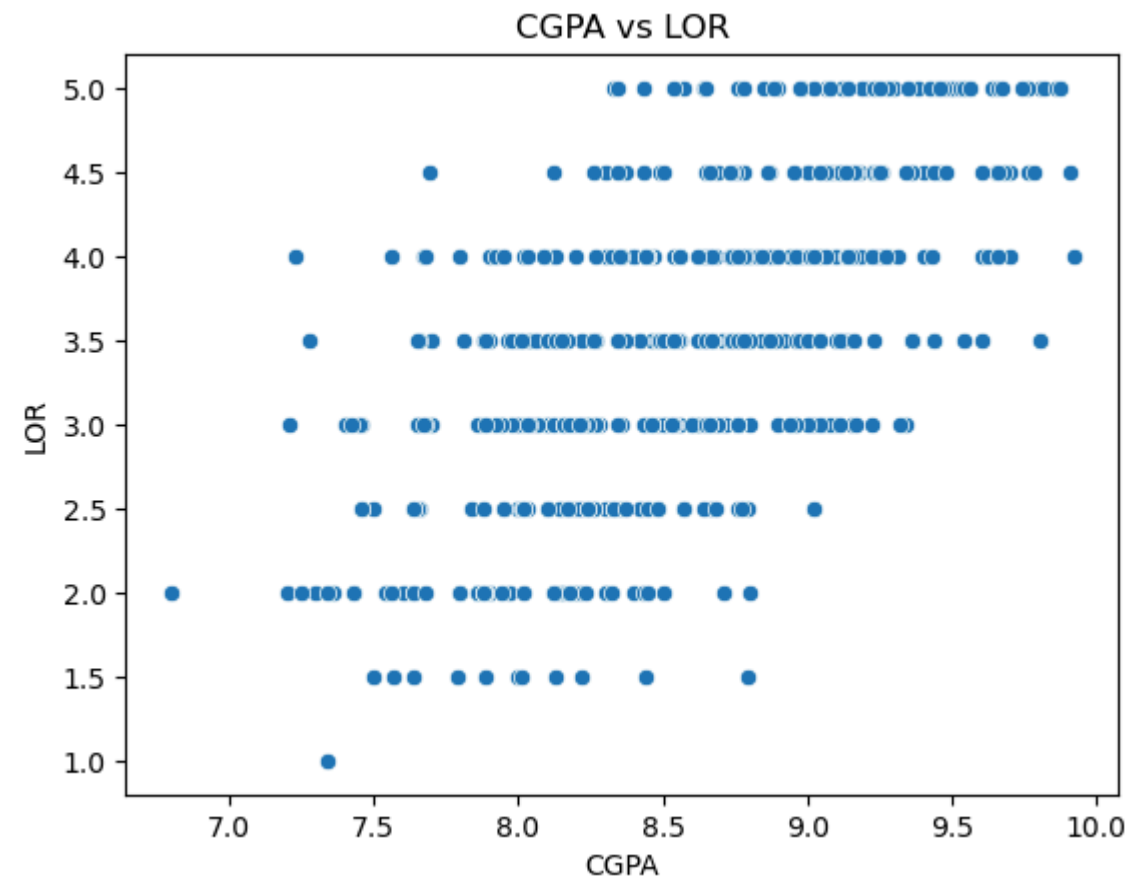


In [17]: # GRE Score Vs CGPA

```
sns.scatterplot(data=df, x='GRE Score', y='CGPA')  
plt.title("GRE Score vs CGPA")  
plt.xlabel("GRE Score")  
plt.ylabel("CGPA")  
plt.show()
```

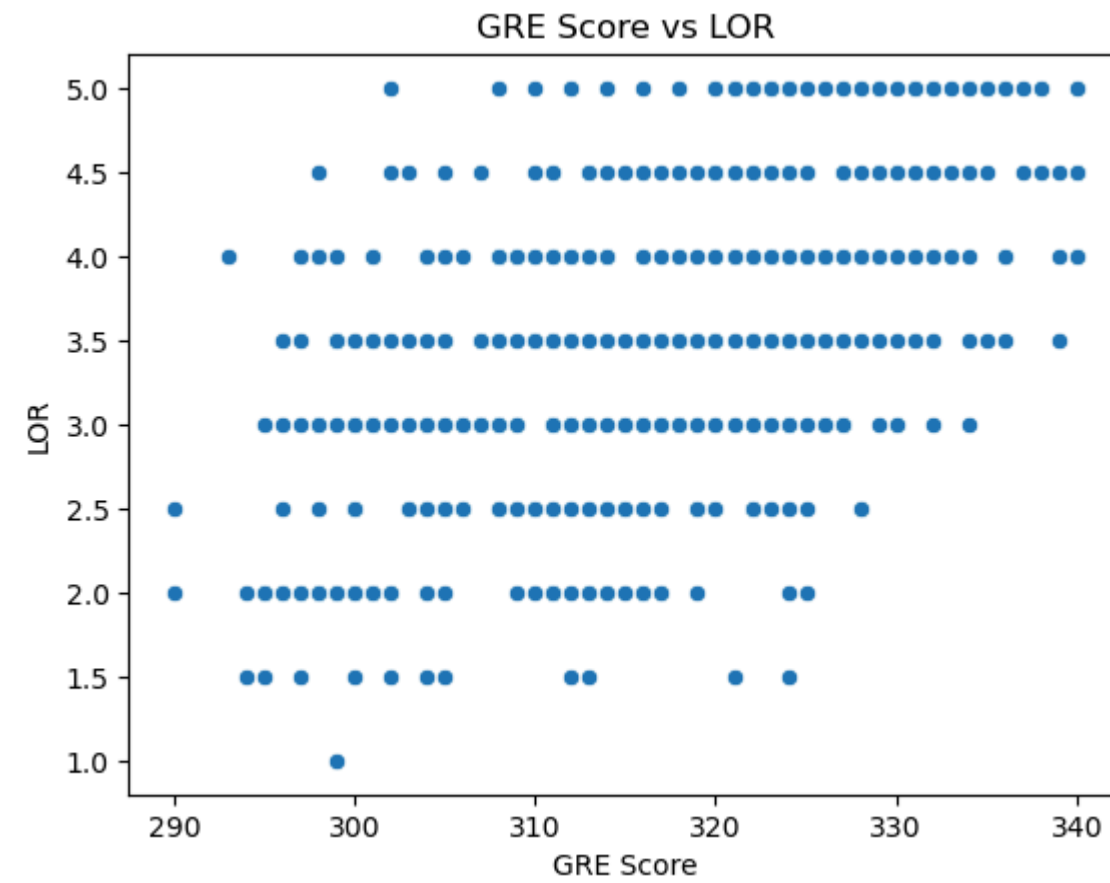


```
sns.scatterplot(data=df, x='CGPA', y='LOR')
plt.title("CGPA vs LOR")
plt.xlabel("CGPA")
plt.ylabel("LOR")
plt.show()
```



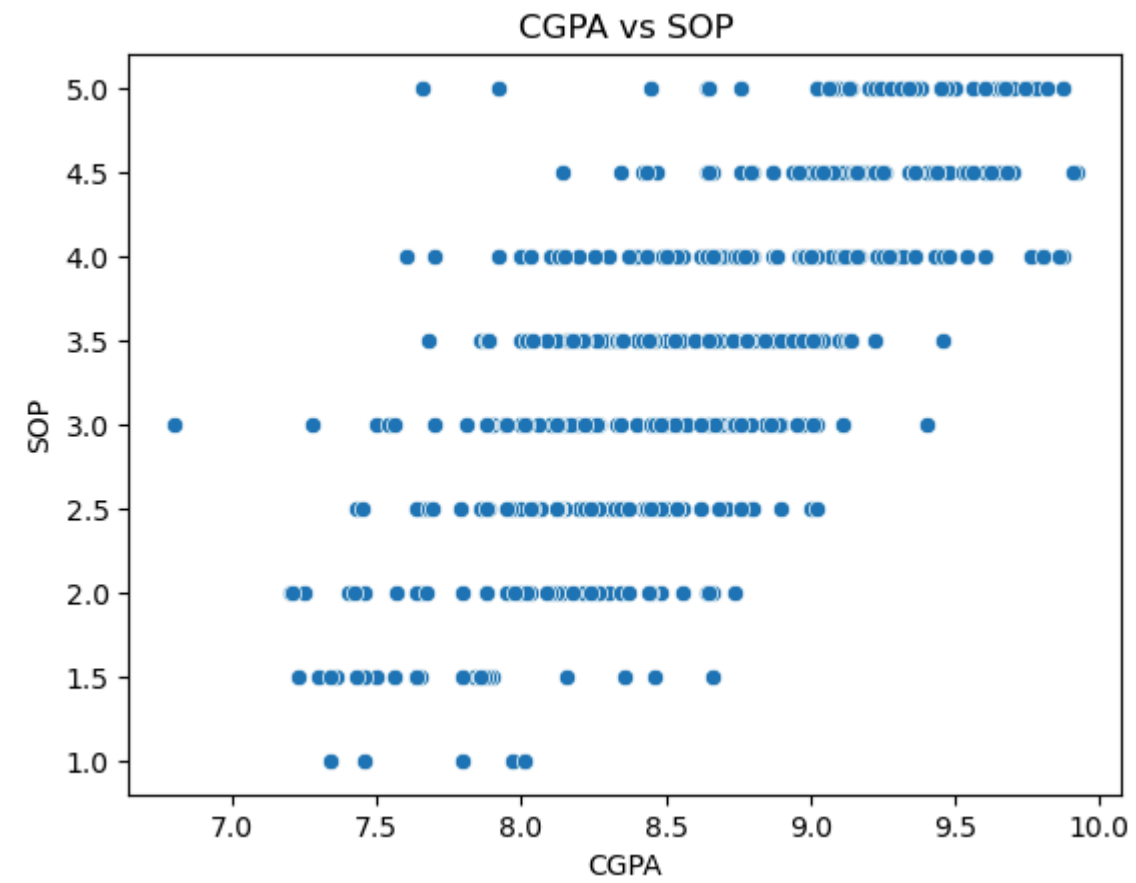
In [19]: # GRE Score Vs LOR

```
sns.scatterplot(data=df, x='GRE Score', y='LOR')  
plt.title("GRE Score vs LOR")  
plt.xlabel("GRE Score")  
plt.ylabel("LOR")  
plt.show()
```



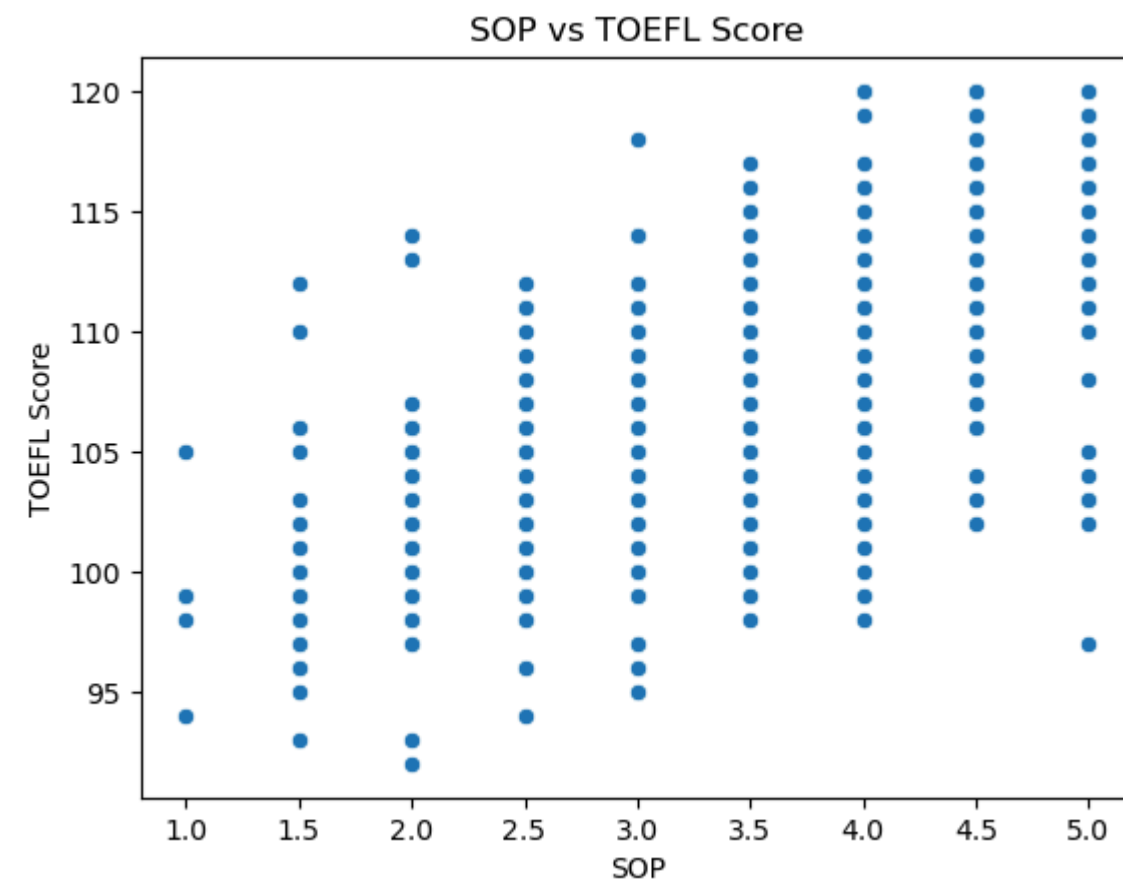
In [20]: # CGPA Vs SOP

```
sns.scatterplot(data=df, x='CGPA', y='SOP')  
plt.title("CGPA vs SOP")  
plt.xlabel("CGPA")  
plt.ylabel("SOP")  
plt.show()
```



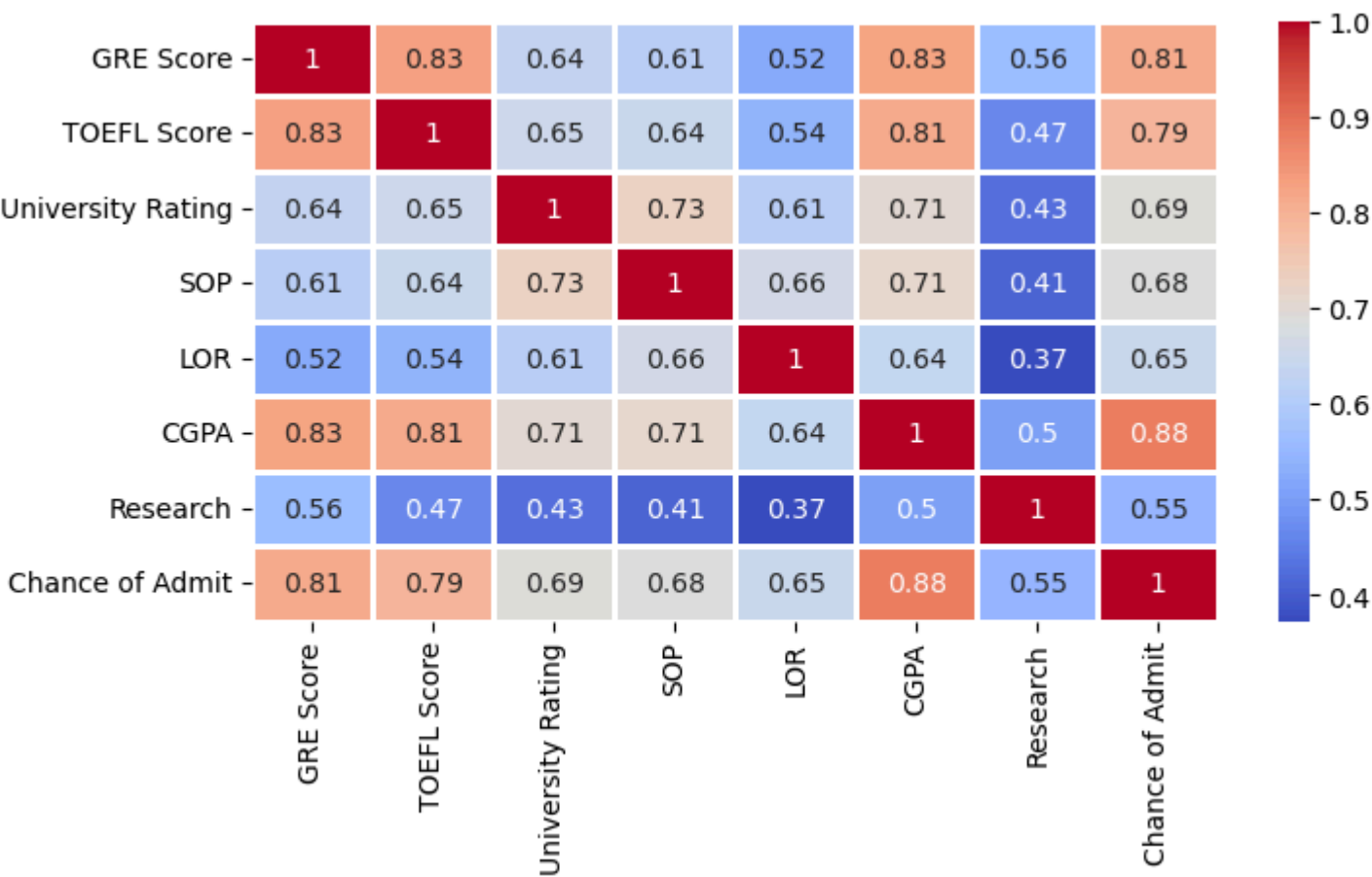
In [21]: # SOP Vs TOEFL

```
sns.scatterplot(data=df, x='SOP', y='TOEFL Score')  
plt.title("SOP vs TOEFL Score")  
plt.xlabel("SOP")  
plt.ylabel("TOEFL Score")  
plt.show()
```



```
In [22]: # now checking the correlation between the features using heatmap
```

```
plt.figure(figsize=(8,4))
corr = df.corr()
sns.heatmap(corr, linewidths=1, annot=True, cmap='coolwarm')
plt.show()
```



Admission Variables Analysis

Key Points

- **CGPA** has the strongest correlation with Chance of Admit (0.88).
- **GRE Score** (0.81) and **TOEFL Score** (0.79) are also highly correlated with admission chances.
- **University Rating** (0.69), **SOP** (0.68), and **LOR** (0.65) have moderate impacts.
- **Research** experience has a smaller, but still positive correlation (0.55).
- All variables are positively correlated with the Chance of Admit.

Conclusion

Focus on improving CGPA, GRE, and TOEFL scores for the best chance of admission. Also, giving attention to university rating, SOP, LOR, and research experience will help improve the overall chances.


```
In [23]: # Computing average admission chance per category

print(df.groupby('University Rating')['Chance of Admit'].mean())
print("\n" + "-"*50 + "\n")
print(df.groupby('Research')['Chance of Admit'].mean())
```

```
University Rating
1    0.562059
2    0.626111
3    0.702901
4    0.801619
5    0.888082
Name: Chance of Admit, dtype: float64

-----

Research
0    0.634909
1    0.789964
Name: Chance of Admit, dtype: float64
```

ML Design: Data Pre-processing

```
In [24]: # duplicate records check

duplicate_recs = df.duplicated().sum()
duplicate_recs
```

Out[24]: 0

All records are unique, no duplicate records present in the dataframe

```
In [25]: # missing values check

df.isna().sum()
```

Out[25]: GRE Score 0
TOEFL Score 0
University Rating 0
SOP 0
LOR 0
CGPA 0
Research 0
Chance of Admit 0
dtype: int64

No missing values present in the dataframe

In [26]: *# checking Outliers using IQR*

```
cont_vars = ['GRE Score', 'TOEFL Score', 'SOP', 'LOR', 'CGPA', 'Chance of Admit']

for var in cont_vars:
    Q1 = df[var].quantile(0.25)
    Q3 = df[var].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5*IQR
    upper_bound = Q3 + 1.5*IQR

    outliers = df[(df[var]<lower_bound) | (df[var]>upper_bound)]
    print(f"{var}: {outliers.shape[0]} outliers")
```

GRE Score: 0 outliers
TOEFL Score: 0 outliers
SOP: 0 outliers
LOR: 1 outliers
CGPA: 0 outliers
Chance of Admit: 2 outliers

No need to treat outliers, since, it is very less in compared to the dataset

In [27]: *# splitting the data into Target and Features*

```
y = df['Chance of Admit']
x = df.drop('Chance of Admit', axis=1)
```

In [28]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size = 0.2, random_state = 42)

In [29]: x_train.shape, x_test.shape, y_train.shape, y_test.shape

Out[29]: ((400, 7), (100, 7), (400,), (100,))

In [30]: *# Scaling the x_train*

```
x_train_columns = x_train.columns

std = StandardScaler()

x_train_std = std.fit_transform(x_train)

x_train = pd.DataFrame(x_train_std, columns=x_train_columns)
```

In [31]: x_train

Out[31]:

	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research
0	0.389986	0.602418	-0.098298	0.126796	0.564984	0.415018	0.895434
1	-0.066405	0.602418	0.775459	0.633979	1.651491	-0.067852	-1.116777
2	-1.253022	-0.876917	-0.098298	0.126796	-0.521524	-0.134454	-1.116777
3	-0.248961	-0.055064	-0.972054	-0.887570	0.564984	-0.517420	-1.116777
4	-0.796631	-0.219435	-0.098298	0.126796	-1.064777	-0.617324	0.895434
...	...	...	...	...	...	...	...
395	1.120212	0.602418	0.775459	1.141162	1.108237	0.997792	0.895434
396	-0.979187	-0.383805	-0.972054	-0.887570	-0.521524	-0.600673	0.895434
397	-1.344300	-1.370029	-1.845810	-1.394754	-1.608031	-2.215790	-1.116777
398	-0.705353	-0.383805	-0.972054	-0.887570	0.564984	-1.499810	-1.116777
399	-0.248961	-0.219435	-0.972054	0.633979	0.021730	-0.550721	-1.116777

400 rows × 7 columns

Regression Models

```
In [32]: # Linear Regression

lr = LinearRegression()
lr.fit(x_train, y_train)

# Predict
y_pred_lr = lr.predict(std.transform(x_test))

# Metrics
r2_lr = r2_score(y_test, y_pred_lr)
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))

print("Linear Regression")
print("-----")
print(f"R2: {r2_lr:.4f}")
print(f"RMSE: {rmse_lr:.4f}")
```

Linear Regression

R2: 0.8188
RMSE: 0.0609

C:\Anaconda\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
warnings.warn(

```
In [33]: # displaying model coefficients with column names

data_lr = list(zip(x_test.columns, lr.coef_))

data_lr = pd.DataFrame(data_lr, columns = ['features', 'coef'])
data_lr
```

Out[33]:

	features	coef
0	GRE Score	0.026671
1	TOEFL Score	0.018226
2	University Rating	0.002940
3	SOP	0.001788
4	LOR	0.015866
5	CGPA	0.067581
6	Research	0.011940

```
In [34]: # Lasso Regression

lasso = Lasso(alpha=0.5)
lasso.fit(x_train, y_train)

y_pred_lasso = lasso.predict(std.transform(x_test))

r2_lasso = r2_score(y_test, y_pred_lasso)
rmse_lasso = np.sqrt(mean_squared_error(y_test, y_pred_lasso))

print("Lasso Regression")
print("-----")
print(f"R2: {r2_lasso:.4f}")
print(f"RMSE: {rmse_lasso:.4f}")

Lasso Regression
-----
R2: -0.0072
RMSE: 0.1435

C:\Anaconda\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid feature names, but Lasso was fitted with feature names
warnings.warn(
```

```
In [35]: # displaying model coefficients with column names

data_lasso = list(zip(x_test.columns, lasso.coef_))

data_lasso = pd.DataFrame(data_lasso, columns = ['features', 'coef'])
data_lasso
```

Out[35]:

	features	coef
0	GRE Score	0.0
1	TOEFL Score	0.0
2	University Rating	0.0
3	SOP	0.0
4	LOR	0.0
5	CGPA	0.0
6	Research	0.0

```
In [36]: # Ridge Regression

ridge = Ridge(alpha=1.0)
ridge.fit(x_train, y_train)

y_pred_ridge = ridge.predict(std.transform(x_test))

r2_ridge = r2_score(y_test, y_pred_ridge)
rmse_ridge = np.sqrt(mean_squared_error(y_test, y_pred_ridge))

print("Ridge Regression")
print("-----")
print(f"R2: {r2_ridge:.4f}")
print(f"RMSE: {rmse_ridge:.4f}")
```

Ridge Regression

R2: 0.8188
RMSE: 0.0609

C:\Anaconda\lib\site-packages\sklearn\base.py:420: UserWarning: X does not have valid feature names, but Ridge was fitted with feature names
warnings.warn(

```
In [37]: # displaying model coefficients with column names

data_ridge = list(zip(x_test.columns, lasso.coef_))

data_ridge = pd.DataFrame(data_ridge, columns = ['features', 'coef'])
data_ridge
```

Out[37]:

	features	coef
0	GRE Score	0.0
1	TOEFL Score	0.0
2	University Rating	0.0
3	SOP	0.0
4	LOR	0.0
5	CGPA	0.0
6	Research	0.0

Ordinary Least Square Model (OLS)


```
In [38]: import statsmodels.api as sm

x_train = sm.add_constant(x_train)

model = sm.OLS(y_train.values, x_train).fit()

print(model.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          y      R-squared:                0.821
Model:                  OLS    Adj. R-squared:           0.818
Method:                 Least Squares    F-statistic:        257.0
Date:                   Fri, 26 Sep 2025    Prob (F-statistic):    3.41e-142
Time:                   19:50:39    Log-Likelihood:       561.91
No. Observations:       400    AIC:                 -1108.
Df Residuals:           392    BIC:                 -1076.
Df Model:                7
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	0.7242	0.003	241.441	0.000	0.718	0.730
GRE Score	0.0267	0.006	4.196	0.000	0.014	0.039
TOEFL Score	0.0182	0.006	3.174	0.002	0.007	0.030
University Rating	0.0029	0.005	0.611	0.541	-0.007	0.012
SOP	0.0018	0.005	0.357	0.721	-0.008	0.012
LOR	0.0159	0.004	3.761	0.000	0.008	0.024
CGPA	0.0676	0.006	10.444	0.000	0.055	0.080
Research	0.0119	0.004	3.231	0.001	0.005	0.019

```
=====
Omnibus:                86.232    Durbin-Watson:           2.050
Prob(Omnibus):           0.000    Jarque-Bera (JB):        190.099
Skew:                    -1.107    Prob(JB):                5.25e-42
Kurtosis:                 5.551    Cond. No.                 5.65
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Testing the assumptions of the Linear Regression Model

1) No Multicollinearity

```
In [39]: from statsmodels.stats.outliers_influence import variance_inflation_factor

def calculate_vif(data,col):
    data=data.drop(columns=col,axis=1)
    vif=pd.DataFrame()
    vif['features']=data.columns
    vif['VIF_Value']=[variance_inflation_factor(data.values,i) for i in range(data.shape[1])]
    return vif
```

```
In [40]: calculate_vif(x_train,[])
```

Out[40]:

	features	VIF_Value
0	const	1.000000
1	GRE Score	4.489983
2	TOEFL Score	3.664298
3	University Rating	2.572110
4	SOP	2.785764
5	LOR	1.977698
6	CGPA	4.654540
7	Research	1.518065

Considering the threshold of 5 to drop the features, all features are below 5, so we can pass this assumption.

```
In [41]: # scaling the x_test data

x_test_std = std.transform(x_test)
x_test = pd.DataFrame(x_test_std, columns=x_train_columns)
```

```
In [42]: x_test
```

0	1.576604	1.424271	0.775459	0.633979	0.021730	1.597217	0.895434
1	-0.248961	0.109306	0.775459	1.141162	0.564984	0.764683	0.895434
2	-0.157683	-0.383805	-0.972054	-1.394754	-1.064777	-1.549762	-1.116777
3	-0.431518	0.273677	-0.098298	-0.380387	-0.521524	0.181909	-1.116777
4	0.846378	0.766789	-0.098298	0.126796	-0.521524	0.781333	0.895434
...	...	...	...	...	...	...	...
95	-1.618135	-2.191882	-1.845810	-2.409120	-2.694538	-2.065934	-1.116777
96	-0.157683	-0.219435	-0.098298	1.141162	0.021730	-0.267660	-1.116777
97	1.120212	1.095530	-0.972054	-1.394754	0.564984	-0.034550	0.895434
98	0.116152	0.438048	-1.845810	-0.887570	0.021730	-0.067852	0.895434
99	-0.248961	-0.383805	-0.098298	0.126796	-1.064777	-0.467468	-1.116777

100 rows × 7 columns


```
In [43]: # adding the constant

x_test = sm.add_constant(x_test)
x_test
```

Out[43]:

	const	GRE Score	TOEFL Score	University Rating	SOP	LOR	CGPA	Research
0	1.0	1.576604	1.424271	0.775459	0.633979	0.021730	1.597217	0.895434
1	1.0	-0.248961	0.109306	0.775459	1.141162	0.564984	0.764683	0.895434
2	1.0	-0.157683	-0.383805	-0.972054	-1.394754	-1.064777	-1.549762	-1.116777
3	1.0	-0.431518	0.273677	-0.098298	-0.380387	-0.521524	0.181909	-1.116777
4	1.0	0.846378	0.766789	-0.098298	0.126796	-0.521524	0.781333	0.895434
...	...	...	...	...	...	...	...	...
95	1.0	-1.618135	-2.191882	-1.845810	-2.409120	-2.694538	-2.065934	-1.116777
96	1.0	-0.157683	-0.219435	-0.098298	1.141162	0.021730	-0.267660	-1.116777
97	1.0	1.120212	1.095530	-0.972054	-1.394754	0.564984	-0.034550	0.895434
98	1.0	0.116152	0.438048	-1.845810	-0.887570	0.021730	-0.067852	0.895434
99	1.0	-0.248961	-0.383805	-0.098298	0.126796	-1.064777	-0.467468	-1.116777

100 rows × 8 columns

```
In [44]: # predicting from the clean model

pred = model.predict(x_test)
pred
```

Out[44]:

0	0.914575
1	0.795181
2	0.572660
3	0.707370
4	0.815883
...	...
95	0.435631
96	0.686643
97	0.785988
98	0.734694
99	0.648657

Length: 100, dtype: float64

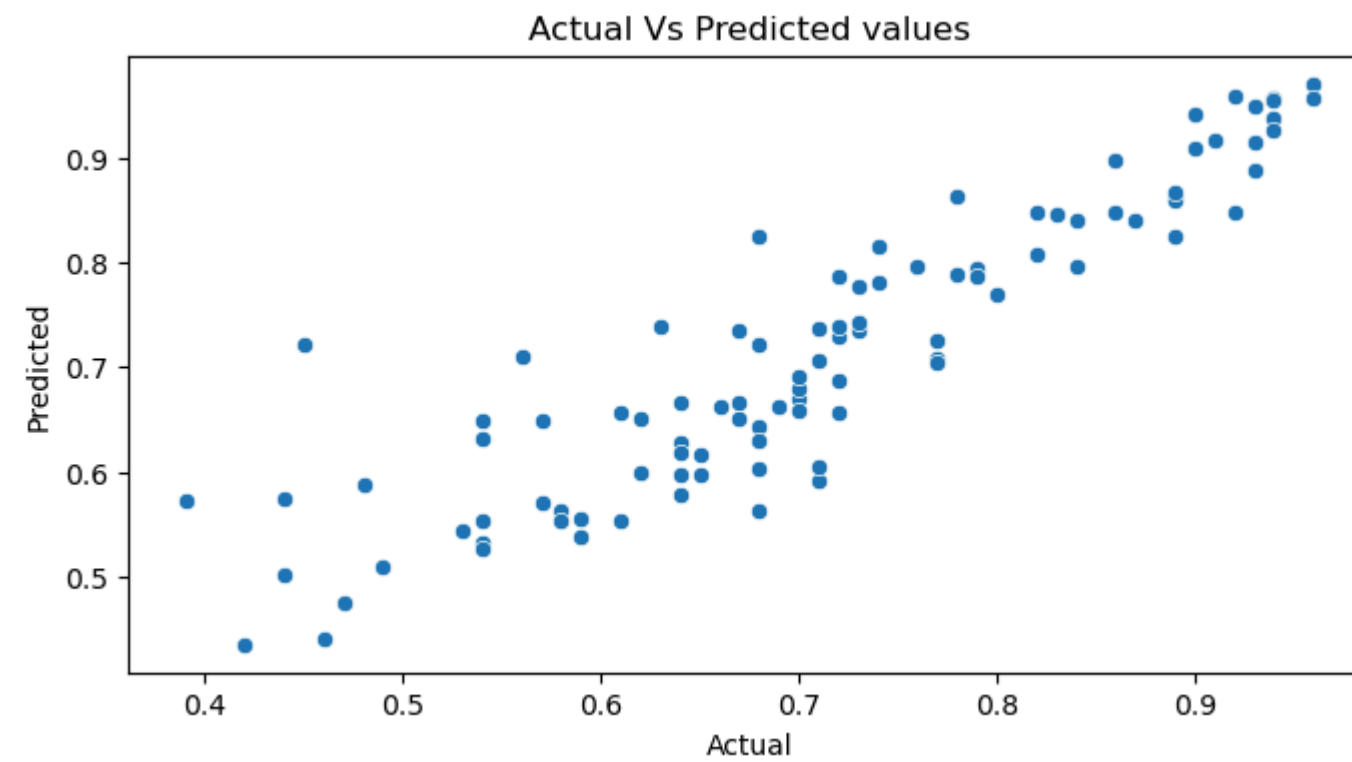
```
In [45]: from sklearn.metrics import mean_squared_error,r2_score,mean_absolute_error

print('Mean Absolute Error:', mean_absolute_error(y_test,pred))
print('Root Mean Square Error:', np.sqrt(mean_squared_error(y_test,pred) ))
print('R² Score:', r2_score(y_test,pred))
```

Mean Absolute Error: 0.042722654277053664
Root Mean Square Error: 0.06086588041578316
R² Score: 0.8188432567829625

2) Assumption of Linearity

```
In [46]: plt.figure(figsize=(8,4))
sns.scatterplot(x=y_test, y=y_pred_lr)
plt.title(f"Actual Vs Predicted values")
plt.xlabel(f"Actual")
plt.ylabel(f"Predicted")
plt.show()
```

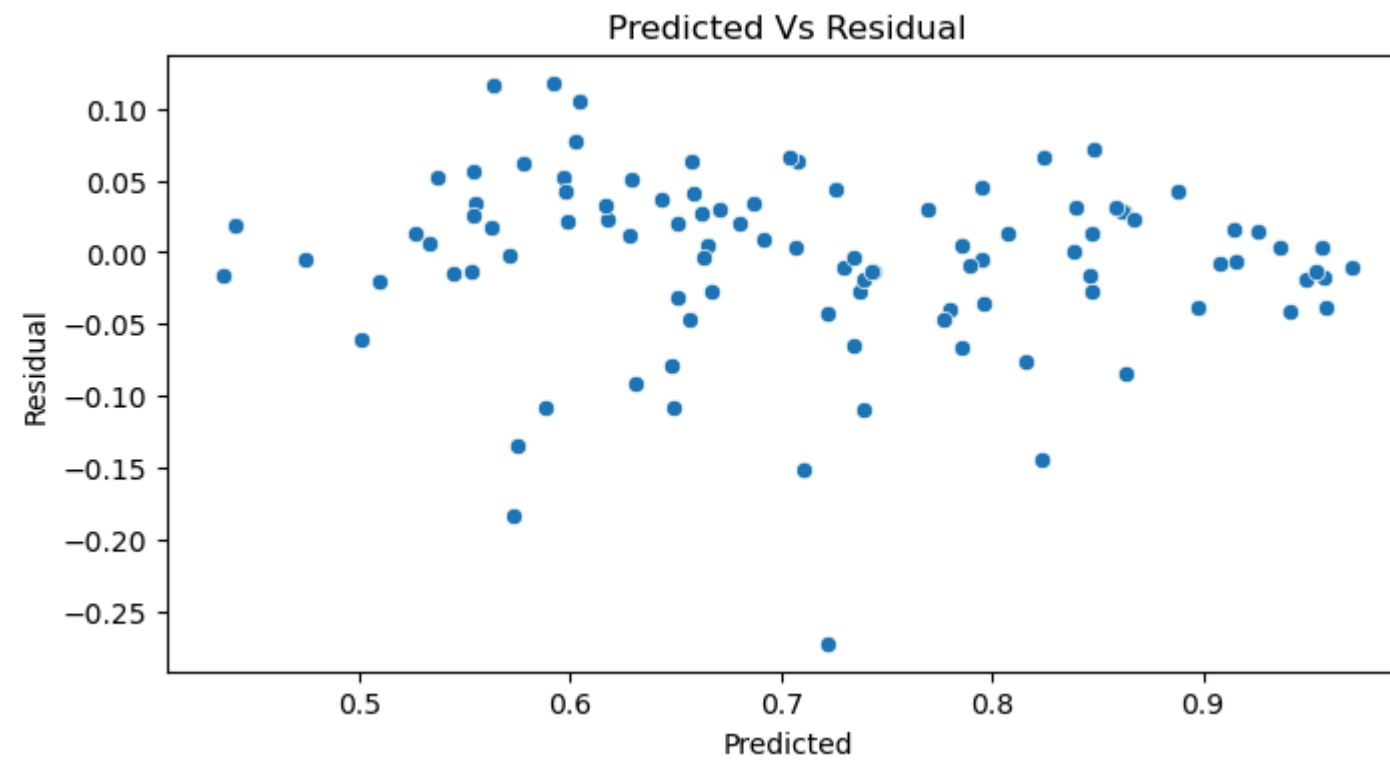


3) Test for Homoscedasticity

Since, the spread is random and not in funnel shape, we can say that the Variance of Residuals is constant.

```
In [47]: # computing residuals
residual = y_test - y_pred_lr

plt.figure(figsize=(8,4))
sns.scatterplot(x=y_pred_lr, y=residual)
plt.title(f"Predicted Vs Residual")
plt.xlabel(f"Predicted")
plt.ylabel(f"Residual")
plt.show()
```



4) Normality of Residuals

As we can notice, both the graphs shows that the residual is close to Normally distributed, that means we can pass the assumption.

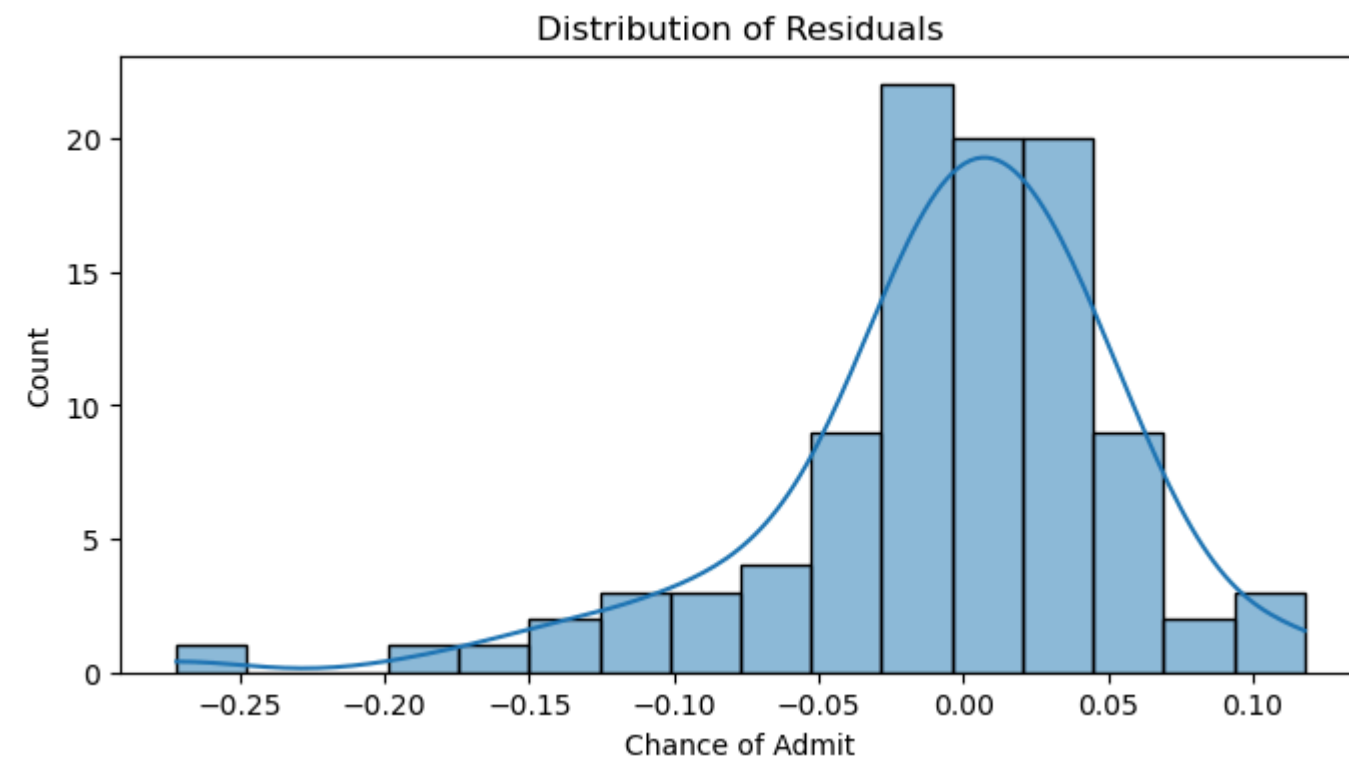
```
In [48]: import scipy.stats as stats
```

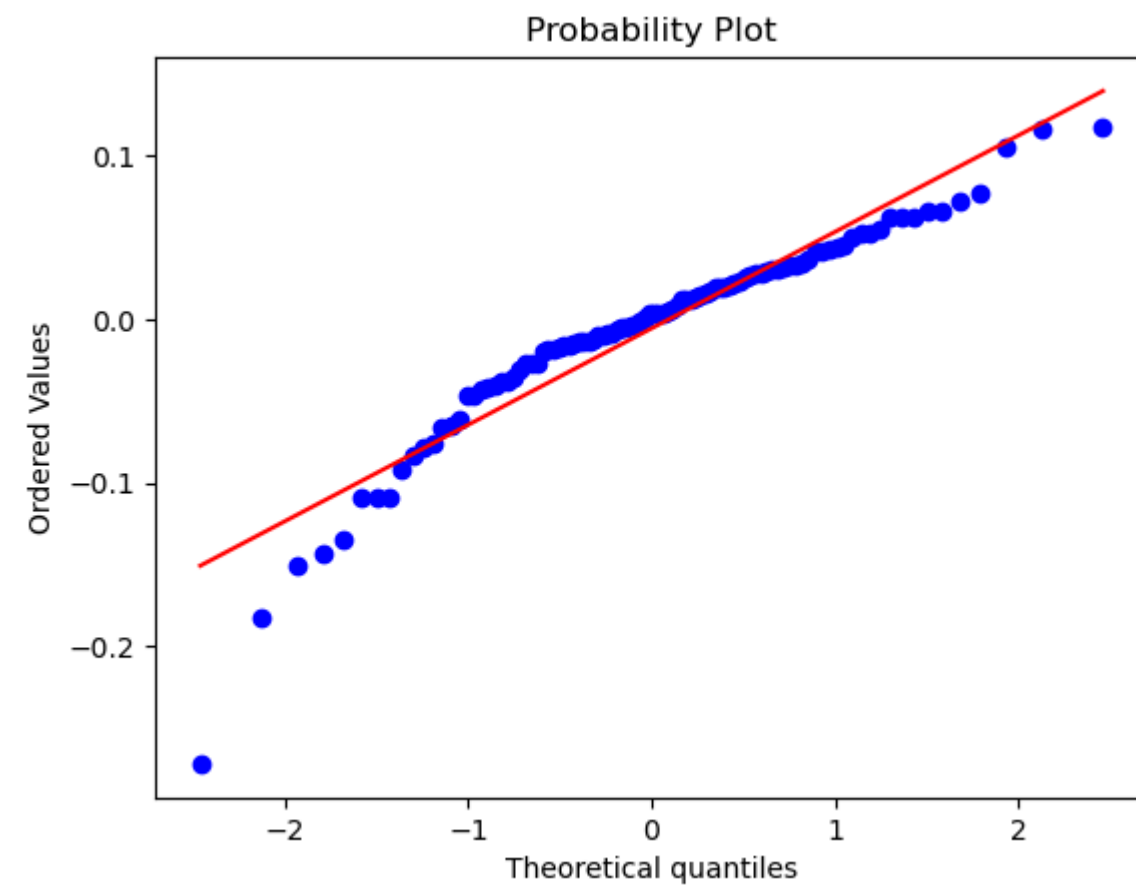
```
# histogram
```

```
plt.figure(figsize=(8,4))  
sns.histplot(residual, kde=True)  
plt.title(f"Distribution of Residuals")  
plt.show()
```

```
# Q-Q plot
```

```
stats.probplot(residual, dist="norm", plot=plt)  
plt.show()
```





5) Mean of Residual close to zero

The mean of residual is almost zero, that means we can also pass this assumption

```
In [49]: residual_mean = np.mean(residual)
print("Mean of residuals:", residual_mean)
```

Mean of residuals: -0.005453623717661255

Feature Recommendations

1. CGPA (Most Important)

- Has the highest impact on admissions.
- Students should focus on **improving their grades** as it gives the best chances.

2. GRE Score

- Plays a strong role, second after CGPA.
- A **good GRE score** increases the admission probability.

3. TOEFL Score

- Also important, but less than GRE and CGPA.
- Students should aim for a **solid TOEFL score** to support their application.

4. LOR (Letter of Recommendation)

- Has a moderate effect.
- Getting **strong recommendations** from professors or supervisors can help.

5. *Research*

- Research experience adds value.
- Doing **projects, papers, or internships** can strengthen the profile.

6. *University Rating*

- Has very little influence.
- The rating of the university applied to matters less than personal scores.

7. *SOP (Statement of Purpose)*

- Least important in the model.
- Still useful, but not a major factor compared to CGPA or GRE.

Final Suggestion

- **Main focus:** Improve **CGPA**, then prepare well for **GRE** and **TOEFL**.
- **Extra support:** Work on good **LORs** and some **research projects**.
- **Less priority:** SOP and University Rating, but they still add a finishing touch.

In []: